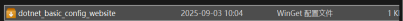


C#基础

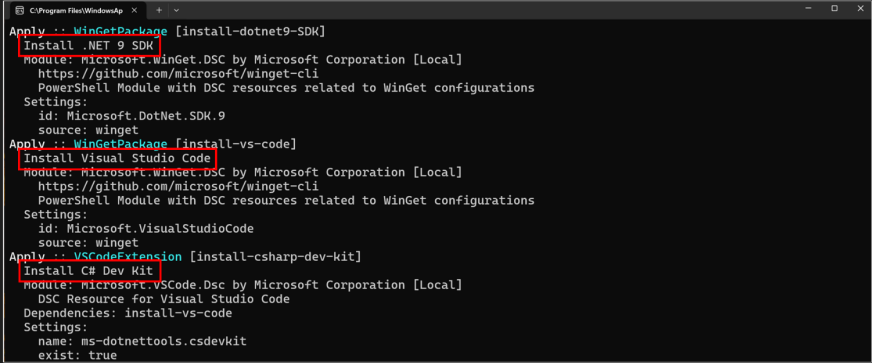
1.VS code环境安装

.net安装:

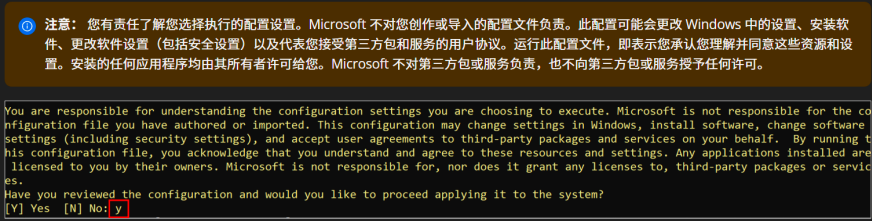
[.NET 教程 - 5 分钟内的 Hello World | .NET](#)



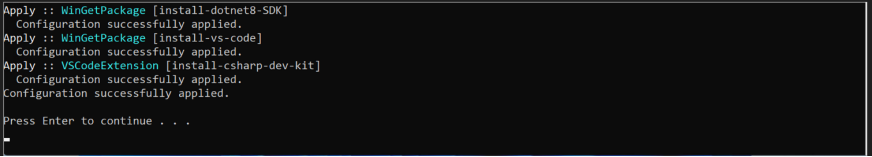
点击安装，跳出终端将看到：



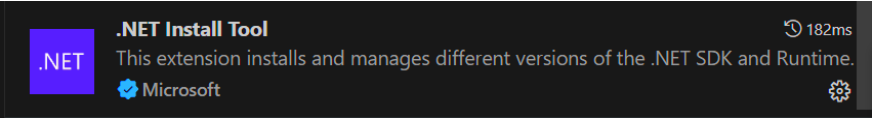
等待结束出现选择:

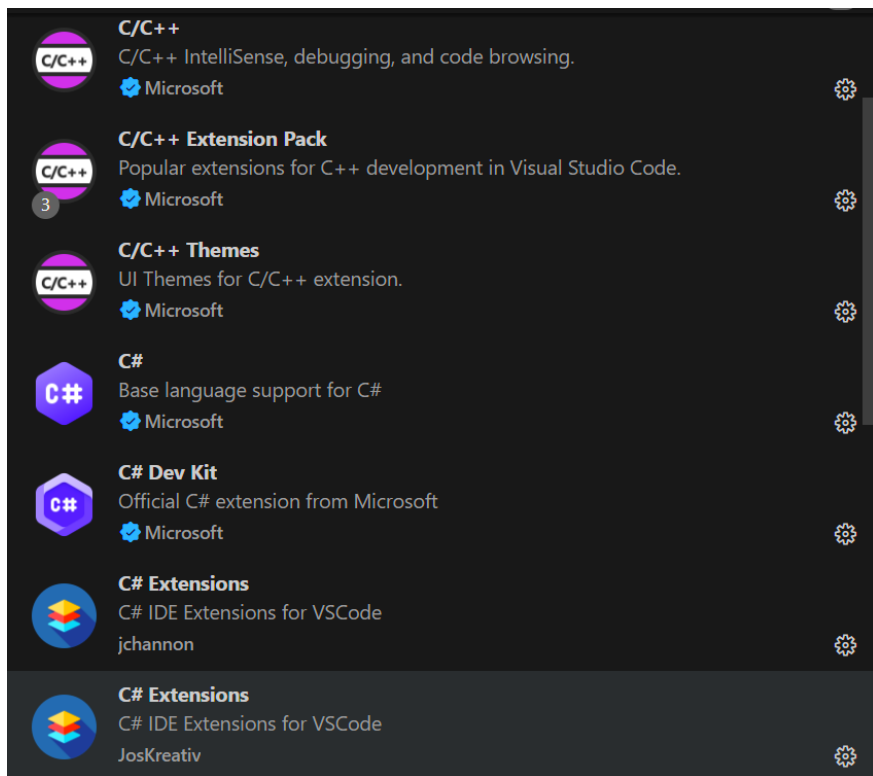


安装完成终端显示:

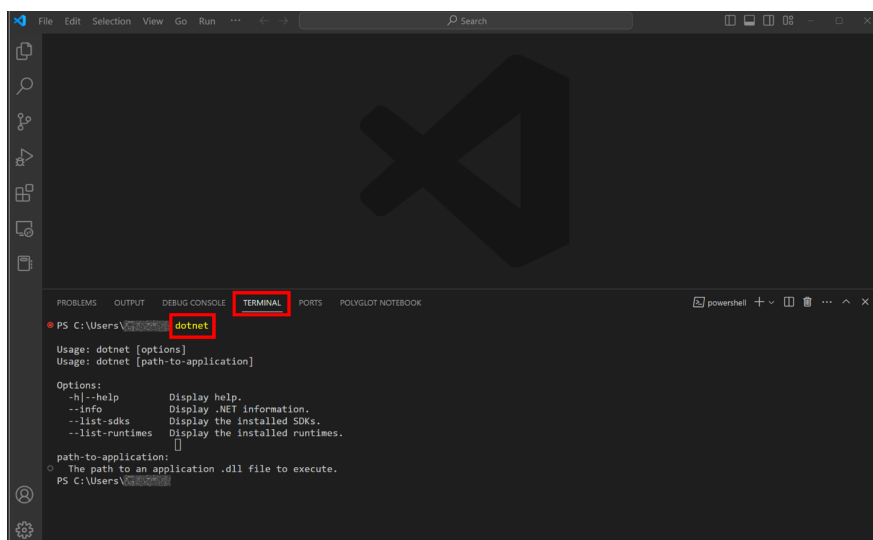


关闭终端重新打开vscode，点击菜单栏的终端，选择新建终端，对应的在扩展中下载插件:





新建终端后，在终端输入：dotnet



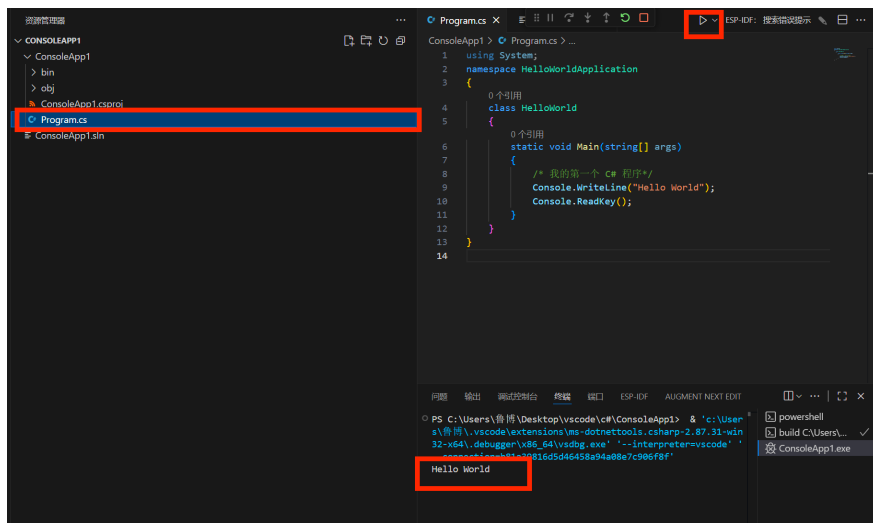
出现图中的信息则说明安装成功。

创建一个新的应用：

1. 按 ++ 在 VS Code 中打开命令面板。CTRL+SHIFT+P
2. 键入 .NET：以查看可以使用 C# 开发工具包运行的命令！
3. 查找并选择 .NET：新建项目以创建新的 .NET 项目。
4. 向下滚动并选择“控制台应用”。
5. 选择要保存项目的文件夹位置。
6. 出现提示时，在命令面板中命名项目。

找到.cs文件，就可以开始写伟大的代码 hello world！

点击运行会在终端出现 hello world



2.C#格式

```
1 一个 C# 程序主要包括以下部分：
2 1.命名空间声明（Namespace declaration）：
3 using System
4 2.一个class
5 class HelloWorld
6 {
7 }
8 3.Class方法
9 static void Main(string[] args)
10 {
11 }
12 4.Class属性（内容）
13 Console.WriteLine("Hello World");
14 5.一个Main方法
15 6.语句（Statements）& 表达式（Expressions）
16 7.注释
```

简单例子：

```
1 using System;
2 namespace HelloWorldApplication
3 {
4     class HelloWorld
5     {
6         static void Main(string[] args)
7         {
8             /* 我的第一个 C# 程序*/
9             Console.WriteLine("Hello World");
10            Console.ReadKey();
11        }
12    }
13 }
```

- 程序的第一行 **using System;** - **using** 关键字用于在程序中包含 **System** 命名空间。一个程序一般有多个 **using** 语句。
- 下一行是 **namespace** 声明。一个 **namespace** 里包含了一系列的类。 *HelloWorldApplication* 命名空间包含了类 *HelloWorld*。
- 下一行是 **class** 声明。类 *HelloWorld* 包含了程序使用的数据和方法声明。类一般包含多个方法。方法定义了类的行为。在这里， *HelloWorld* 类只有一个 **Main** 方法。
- 下一行定义了 **Main** 方法，是所有 C# 程序的 **入口点**。 **Main** 方法说明当执行时 类将做什么动作。
- 下一行 */*...*/* 将会被编译器忽略，且它会在程序中添加额外的 **注释**。

- Main 方法通过语句 **Console.WriteLine("Hello World");** 指定了它的行为。*WriteLine* 是一个定义在 *System* 命名空间中的 *Console* 类的一个方法。该语句会在屏幕上显示消息 "Hello World"。
- 最后一行 **Console.ReadKey();** 是针对 VS.NET 用户的。这使得程序会等待一个按键的动作，防止程序从 Visual Studio .NET 启动时屏幕会快速运行并关闭。

特别注意以下几点：

- C# 是**大小写敏感**的。
- 所有的语句和表达式必须以分号 (;) 结尾。
- 程序的执行从 Main 方法开始。
- 与 Java 不同的是，文件名可以不同于类的名称。

3.C#基本语法

C# 是一种面向对象的编程语言。在面向对象的程序设计方法中，程序由各种相互交互的对象组成。相同种类的对象通常具有相同的类型，或者说，是在相同的 class 中。例如，以 Rectangle（矩形）对象为例。它具有 length 和 width 属性。根据设计，它可能需要接受这些属性值、计算面积和显示细节。

```
1  using System;
2
3  // 独立命名空间，与HelloWorldApplication区分
4  namespace RectangleApplication
5  {
6      class Rectangle
7      {
8          // 成员变量
9          double length;
10         double width;
11
12         public void Acceptdetails()
13         {
14             length = 4.5;
15             width = 3.5;
16         }
17
18         public double GetArea()
19         {
20             return length * width;
21         }
22
23         public void Display()
24         {
25             Console.WriteLine("Length: {0}", length);
26             Console.WriteLine("Width: {0}", width);
27             Console.WriteLine("Area: {0}", GetArea());
28         }
29     }
30
31     // 独立的入口类，包含Main方法
32     class RectangleProgram
33     {
34         static void Main(string[] args)
35         {
36             Rectangle r = new Rectangle();
37             r.Acceptdetails();
38             r.Display();
39             Console.ReadLine(); // 等待输入，防止控制台闪退
40         }
41     }
42 }
```

using关键字

在任何 C# 程序中的第一条语句都是：

using System;

using 关键字用于在程序中包含命名空间。一个程序可以包含多个 using 语句

class关键字

class 关键字用于声明一个类

C#中的注释

多行注释: /**/
单行注释: //

成员变量

变量是类的属性或者是数据成员，用于存储数据

成员函数

函数是一系列执行指定任务的语句。
类的成员函数是在类内声明的。
我们举例的类 Rectangle 包含了三个成员函数： *AcceptDetails*、*GetArea* 和 *Display*。

实例化一个类

类 *ExecuteRectangle* 是一个包含 *Main()* 方法和实例化 *Rectangle* 类的类。

标识符

- 标识符是用来识别类、变量、函数或任何其它用户定义的项目。遵循以下规则：
- 标识符必须以字母、下划线或 @ 开头，后面可以跟一系列的字母、数字（0 - 9）、下划线（_）、@。
 - 标识符中的第一个字符不能是数字。
 - 标识符必须不包含任何嵌入的空格或符号，比如 ? - + ! # % ^ & * () [] { } . ; : " ' / \。
 - 标识符不能是 C# 关键字。除非它们有一个 @ 前缀。例如，@if 是有效的标识符，但 if 不是，因为 if 是关键字。
 - 标识符必须区分大小写。大写字母和小写字母被认为是不同的字母。
 - 不能与C#的类库名称相同。

C#关键字

关键字不能作为标识符，如果想把他作为关键字就在关键字前面加@字符作为前缀
C#中，有些关键字比较特殊，get和set被称为上下文关键字。

保留关键字						
abstract	as	base	bool	break	byte	case
catch	char	checked	class	const	continue	decimal
default	delegate	do	double	else	enum	event
explicit	extern	false	finally	fixed	float	for
foreach	goto	if	implicit	in	in (generic modifier)	int
interface	internal	is	lock	long	namespace	new
null	object	operator	out	out (generic modifier)	override	params
private	protected	public	readonly	ref	return	sbyte
sealed	short	sizeof	stackalloc	static	string	struct
switch	this	throw	true	try	typeof	uint
ulong	unchecked	unsafe	ushort	using	virtual	void
volatile	while					

上下文关键字						
add	alias	ascending	descending	dynamic	from	get
global	group	into	join	let	orderby	partial (type)
partial (method)	remove	select	set			

顶级语句（Top-Level Statements）

在C#9.0版本引入了顶级语句，这个范式运行开发者在文件的顶层直接编写代码，不需要封装在方法或类里

注意：

- **文件限制：**顶级语句只能在一个源文件中使用。如果在一个项目中有多个使用顶级语句的文件，会导致编译错误。
- **程序入口：**如果使用顶级语句，则该文件会隐式地包含 Main 方法，并且该文件将成为程序的入口点。
- **作用域限制：**顶级语句中的代码共享一个全局作用域，这意味着可以在顶级语句中定义的变量和方法可以在整个文件中访问。

特点：

- **无需类或方法：**顶级语句允许你直接在文件的顶层编写代码，无需定义类或方法。
- **文件作为入口点：**包含顶级语句的文件被视为程序的入口点，类似于 C# 之前的 `Main` 方法。
- **自动 `Main` 方法：**编译器会自动生成一个 `Main` 方法，并将顶级语句作为 `Main` 方法的主体。
- **支持局部函数：**尽管不需要定义类，但顶级语句的文件中仍然可以定义局部函数。
- **更好的可读性：**对于简单的脚本或工具，顶级语句提供了更好的可读性和简洁性。
- **适用于小型项目：**顶级语句非常适合小型项目或脚本，可以快速编写和运行代码。
- **与现有代码兼容：**顶级语句可以与现有的 C# 代码库一起使用，不会影响现有代码。

例如：

```
using System;
Console.WriteLine("Hello, World!");
```

```
1 using System;
2 using System.Linq;
3
4 // 顶级语句中的变量声明
5 int number = 42;
6 string message = "The answer to life, the universe, and everything is";
7
8 // 输出变量
9 Console.WriteLine($"{message} {number}.");
10
11 // 定义和调用方法
12 int Add(int a, int b) => a + b;
13 Console.WriteLine($"Sum of 1 and 2 is {Add(1, 2)}.");
14
15 // 使用 LINQ
16 var numbers = new[] { 1, 2, 3, 4, 5 };
17 var evens = numbers.Where(n => n % 2 == 0).ToArray();
18 Console.WriteLine($"Even numbers: " + string.Join(", ", evens));
19
20 // 异常处理
21 try
22 {
23     int zero = 0;
24     int result = number / zero;
25 }
26 catch (DivideByZeroException ex)
27 {
28     Console.WriteLine($"Error: " + ex.Message);
29 }
```

4.C#数据类型

值类型（Value types）

值类型变量可以直接分配给一个值，他们是从类System.ValueType中派生
值类型直接包含数据。比如 `int`、`char`、`float`，它们分别存储数字、字符、浮点数。当您声明一个 `int` 类型时，系统分配内存来存储值。

类型	描述	范围	默认值
bool	布尔值	True 或 False	False
byte	8 位无符号整数	0 到 255	0
char	16 位 Unicode 字符	U +0000 到 U +ffff	'\0'
decimal	128 位精确的十进制值，28-29 有效位数	(-7.9 x 10 ²⁸ 到 7.9 x 10 ²⁸) / 10 ⁰ 到 28	0.0M
double	64 位双精度浮点型	(+/-)5.0 x 10 ⁻³²⁴ 到 (+/-)1.7 x 10 ³⁰⁸	0.0D
float	32 位单精度浮点型	-3.4 x 10 ³⁸ 到 + 3.4 x 10 ³⁸	0.0F
int	32 位有符号整数类型	-2,147,483,648 到 2,147,483,647	0
long	64 位有符号整数类型	-9,223,372,036,854,775,808 到 9,223,372,036,854,775,807	0L
sbyte	8 位有符号整数类型	-128 到 127	0
short	16 位有符号整数类型	-32,768 到 32,767	0
uint	32 位无符号整数类型	0 到 4,294,967,295	0
ulong	64 位无符号整数类型	0 到 18,446,744,073,709,551,615	0
ushort	16 位无符号整数类型	0 到 65,535	0

表达式 `sizeof(type)` 产生以字节为单位存储对象或类型的存储尺寸。

引用类型（Reference types）

引用类型不包含存储在变量中的实际数据，但它们包含对变量的引用
它们指的是一个内存位置。使用多个变量时，引用类型可以指向一个内存位置。如果内存位置的数据是由一个变量改变的，其他变量会自动反映这种值的变化。**内置的** 引用类型有：`object`、`dynamic` 和 `string`。

对象（Object）类型

对象（Object）类型 是 C# 通用类型系统中所有数据类型的终极基类。

`Object` 是 `System.Object` 类的别名。

所以对象（Object）类型可以被分配任何其他类型（值类型、引用类型、预定义类型或用户自定义类型）的值。但是，在分配值之前，需要先进行类型转换。
当一个值类型转换为对象类型时，则被称为 **装箱**；另一方面，当一个对象类型转换为值类型时，则被称为 **拆箱**。

```
object obj;  
obj = 100; // 这是装箱
```

动态（Dynamic）类型

声明动态类型的语法：

```
1 dynamic <variable_name> = value;  
2 例如：  
3 dynamic d = 20;
```

动态类型与对象类型相似，
对象类型变量的类型检查是在**编译时发生的**，
动态类型变量的类型检查是在**运行时发生的**。

字符串（String）类型

字符串（String）类型 允许给变量分配任何字符串值。

字符串（String）类型是 System.String 类的别名。它是从对象（Object）类型派生的。

字符串（String）类型的值可以通过两种形式进行分配：引号和 @引号。

例如：

```
1 String str = "runoob.com";
```

一个 @引号字符串：

```
1 @"runoob.com";
```

C# string 字符串的前面可以加 @（称作“逐字符串”）将转义字符（\）当作普通字符对待，比如：

```
1 string str = @"C:\Windows";
```

等价于：

```
1 string str = "C:\\Windows";
```

@ 字符串中可以任意换行，换行符及缩进空格都计算在字符串长度之内。

```
1 string str = @"<script type=""text/javascript"">
2     <!--
3     -->
4 </script>";
```

用户自定义引用类型有：class、interface 或 delegate。我们将在以后的章节中讨论这些类型。

指针类型（Pointer types）

指针类型变量存储另一种类型的内存地址。C# 中的指针与 C 或 C++ 中的指针有相同的功能。

声明指针类型的语法：

```
1 type* identifier;
```

例如：

```
1 char* cptr;
2 int* iptr;
```

5.C#类型转换

隐式类型转换

隐式转换是不需要编写代码来指定的转换，编译器会自动进行。

隐式转换是指将一个较小范围的数据类型转换为较大范围的数据类型时，编译器会自动完成类型转换，这些转换是 C# 默认的以安全方式进行的转换，不会导致数据丢失。

例如，从 int 到 long，从 float 到 double 等。

从小的整数类型转换为大的整数类型，从派生类转换为基类。将一个 byte 类型的变量赋值给 int 类型的变量，编译器会自动将 byte 类型转换为 int 类型，不需要显示转换。

实例

```
byte b = 10;
```


`int i = b;` // 隐式转换, 不需要显式转换

将一个整数赋值给一个长整数, 或者将一个浮点数赋值给一个双精度浮点数, 这种转换不会导致数据丢失:

实例

`int intValue = 42;`

`long longValue = intValue;` // 隐式转换, 从 `int` 到 `long`

显式转换

显式类型转换, 即强制类型转换, 需要程序员在代码中明确指定。

显式转换是指将一个较大范围的数据类型转换为较小范围的数据类型时, 或者将一个对象类型转换为另一个对象类型时, 需要使用强制类型转换符号进行显示转换, 强制转换会造成数据丢失。

例如, 将一个 `int` 类型的变量赋值给 `byte` 类型的变量, 需要显示转换。

强制转换为整数:

`double doubleValue = 3.14;`

`int intValue = (int)doubleValue;` // 强制从 `double` 到 `int`, 数据可能损失小数部分

强制转换为浮点数类型:

`int intValue = 42;`

`float floatValue = (float)intValue;` // 强制从 `int` 到 `float`, 数据可能损失精度

强制转换为字符串类型:

`int intValue = 123;`

`string stringValue = intValue.ToString();` // 将 `int` 转换为字符串

下面的实例显示了一个显式的类型转换:

实例

```
1 using System;
2
3 namespace TypeConversionApplication
4 {
5     class ExplicitConversion
6     {
7         static void Main(string[] args)
8         {
9             double d = 5673.74;
10            int i;
11
12            // 强制转换 double 为 int
13                i = (int)d;
14                Console.WriteLine(i);
15                Console.ReadKey();
16
17        }
18    }
19 }
20 结果:
21 5673
```

C# 类型转换方法

C# 提供了下列内置的类型转换方法:

序号	方法 & 描述
1	ToBoolean 如果可能的话，把类型转换为布尔型。
2	ToByte 把类型转换为字节类型。
3	ToChar 如果可能的话，把类型转换为单个 Unicode 字符类型。
4	ToDateTime 把类型（整数或字符串类型）转换为 日期-时间 结构。
5	ToDecimal 把浮点型或整数类型转换为十进制类型。
6	ToDouble 把类型转换为双精度浮点型。
7	ToInt16 把类型转换为 16 位整数类型。
8	ToInt32 把类型转换为 32 位整数类型。
9	ToInt64 把类型转换为 64 位整数类型。
10	ToSbyte 把类型转换为有符号字节类型。
11	ToSingle 把类型转换为小浮点数类型。
12	ToString 把类型转换为字符串类型。
13	ToType 把类型转换为指定类型。
14	ToUInt16 把类型转换为 16 位无符号整数类型。
15	ToUInt32 把类型转换为 32 位无符号整数类型。
16	ToUInt64 把类型转换为 64 位无符号整数类型。

:

```
1 using System;
2
3 namespace TypeConversionApplication
4 {
5     class StringConversion
6     {
7         static void Main(string[] args)
8         {
9             // 定义一个整型变量
10             int i = 75;
11
12             // 定义一个浮点型变量
13             float f = 53.005f;
14 }
```

```
15         // 定义一个双精度浮点型变量
16         double d = 2345.7652;
17
18         // 定义一个布尔型变量
19         bool b = true;
20
21         // 将整型变量转换为字符串并输出
22         Console.WriteLine(i.ToString());
23
24         // 将浮点型变量转换为字符串并输出
25         Console.WriteLine(f.ToString());
26
27         // 将双精度浮点型变量转换为字符串并输出
28         Console.WriteLine(d.ToString());
29
30         // 将布尔型变量转换为字符串并输出
31         Console.WriteLine(b.ToString());
32
33         // 等待用户按键后关闭控制台窗口
34         Console.ReadKey();
35     }
36 }
37 }
38 结果:
39 75
40 53.005
41 2345.7652
42 True
```

注意:

- 隐式转换只能将较小范围的数据类型转换为较大范围的数据类型，不能将较大范围的数据类型转换为较小范围的数据类型；
- 显式转换可能会导致数据丢失或精度降低，需要进行数据类型的兼容性检查；
- 对于对象类型的转换，需要进行类型转换的兼容性检查和类型转换的安全性检查。

6.C#变量

C# 中变量定义的语法:

```
1  <data_type> <variable_list>;
2  例如:
3  int i, j, k;
4  char c, ch;
5  float f, salary;
6  double d;
7  初始化:
8  int i = 100;
9  初始化实例:
10 int d = 3, f = 5;    /* 初始化 d 和 f. */
11 byte z = 22;        /* 初始化 z. */
12 double pi = 3.14159; /* 声明 pi 的近似值 */
13 char x = 'x';        /* 变量 x 的值为 'x' */
```

变量的命名规则

在 C# 中，变量的命名需要遵循一些规则：

- 变量名可以包含字母、数字和下划线。
- 变量名必须以字母或下划线开头。
- 变量名区分大小写。
- 避免使用 C# 的关键字作为变量名。

C# 中的 Lvalues 和 Rvalues

C# 中的两种表达式：

- 1. **lvalue**：lvalue 表达式可以出现在赋值语句的左边或右边。
 - 2. **rvalue**：rvalue 表达式可以出现在赋值语句的右边，不能出现在赋值语句的左边。
- 变量是 lvalue 的，所以可以出现在赋值语句的左边。数值是 rvalue 的，因此不能被赋值，不能出现在赋值语句的左边。

7.C#变量作用域

变量的作用域定义了变量的可见性和生命周期。
变量的作用域通常由花括号 {} 定义的代码块来确定。
以下是关于C#变量作用域的一些基本规则：

局部变量

在方法、循环、条件语句等代码块内声明的变量是局部变量，它们只在声明它们的代码块中可见。

实例

```
void MyMethod()  
{  
    int localVar = 10; // 局部变量  
    // ...  
}  
// localVar 在这里不可见
```

块级作用域

在 C# 7及更高版本中，引入了块级作用域，即使用大括号 {} 创建的任何块都可以定义变量的作用域。

实例{

```
int blockVar = 20; // 块级作用域  
// ...  
}  
// blockVar 在这里不可见
```

方法参数作用域

方法的参数也有其自己的作用域，它们在整个方法中都是可见的。

实例

```
void MyMethod(int parameter)  
{  
    // parameter 在整个方法中可见  
    // ...  
}
```

全局变量

在类的成员级别定义的变量是成员变量，它们在整个类中可见，如果在命名空间级别定义，那么它们在整个命名空间中可见。

实例

```
class MyClass  
{  
    int memberVar = 30; // 成员变量，在整个类中可见  
}
```

静态变量作用域

静态变量是在类级别上声明的，但它们的作用域也受限于其定义的类。

实例

```
class MyClass  
{  
    static int staticVar = 40; // 静态变量，在整个类中可见  
}
```

循环变量作用域

在 for 循环中声明的循环变量在循环体内可见。

实例

```
for (int i = 0; i < 5; i++)  
{  
    // i 在循环体内可见
```

```
}  
// i 在这里不可见
```

总体而言，变量的作用域有助于管理变量的可见性和生命周期，确保变量在其有效范围内使用，也有助于防止命名冲突。

8.C#常量

字符常量

转义序列	含义
\\	\ 字符
'\'	' 字符
\"	" 字符
\?	? 字符
\a	Alert 或 bell
\b	退格键（Backspace）
\f	换页符（Form feed）
\n	换行符（Newline）
\r	回车
\t	水平制表符 tab
\v	垂直制表符 tab
\ooo	一到三位的八进制数
\xhh...	一个或多个数字的十六进制数

字符串常量

字符串常量是括在双引号 "" 里，或者是括在 @"" 里。字符串常量包含的字符与字符常量相似，可以是：普通字符、转义序列和通用字符

使用字符串常量时，可以把一个很长的行拆成多个行，可以使用空格分隔各个部分。

这里是一些字符串常量的实例。下面所列的各种形式表示相同的字符串。

```
1 string a = "hello, world";           // hello, world  
2 string b = @"hello, world";         // hello, world  
3 string c = "hello \t world";        // hello    world  
4 string d = @"hello \t world";       // hello \t world  
5 string e = "Joe said \"Hello\" to me"; // Joe said "Hello" to me  
6 string f = @"Joe said ""Hello"" to me"; // Joe said "Hello" to me  
7 string g = "\\server\\share\\file.txt"; // \\server\share\file.txt  
8 string h = @"\\server\share\file.txt"; // \\server\share\file.txt  
9 string i = "one\r\ntwo\r\nthree";  
10 string j = @"one  
11 two  
12 three";
```

定义常量

常量是使用 **const** 关键字来定义的。定义一个常量的语法如下：

```
1 const <data_type> <constant_name> = value;  
2 //实例：  
3 using System;  
4  
5 public class ConstTest  
6 {  
7     class SampleClass
```

```
8      {
9          public int x;
10         public int y;
11         public const int c1 = 5;
12         public const int c2 = c1 + 5;
13
14         public SampleClass(int p1, int p2)
15         {
16             x = p1;
17             y = p2;
18         }
19     }
20
21     static void Main()
22     {
23         SampleClass mC = new SampleClass(11, 22);
24         Console.WriteLine("x = {0}, y = {1}", mC.x, mC.y);
25         Console.WriteLine("c1 = {0}, c2 = {1}",
26                             SampleClass.c1, SampleClass.c2);
27     }
28 }
29 结果:
30 x = 11, y = 22
31 c1 = 5, c2 = 10
```

9.C#运算符

运算符是一种告诉编译器执行特定的数学或逻辑操作的符号。C# 有丰富的内置运算符，分类如下：

- 算术运算符
- 关系运算符
- 逻辑运算符
- 位运算符
- 赋值运算符
- 其他运算符

算术运算符

下表显示了 C# 支持的所有算术运算符。假设变量 **A** 的值为 10，变量 **B** 的值为 20，则：

运算符	描述	实例
+	把两个操作数相加	A + B 将得到 30
-	从第一个操作数中减去第二个操作数	A - B 将得到 -10
*	把两个操作数相乘	A * B 将得到 200
/	分子除以分母	B / A 将得到 2
%	取模运算符，整除后的余数	B % A 将得到 0
++	自增运算符，整数值增加 1	A++ 将得到 11
--	自减运算符，整数值减少 1	A-- 将得到 9

关系运算符

下表显示了 C# 支持的所有关系运算符。假设变量 **A** 的值为 10，变量 **B** 的值为 20，则：

运算符	描述	实例
==	检查两个操作数的值是否相等，如果相等则条件为真。	(A == B) 不为真。
!=	检查两个操作数的值是否相等，如果不相等则条件为真。	(A != B) 为真。
>	检查左操作数的值是否大于右操作数的值，如果是则条件为真。	(A > B) 不为真。
<	检查左操作数的值是否小于右操作数的值，如果是则条件为真。	(A < B) 为真。
>=	检查左操作数的值是否大于或等于右操作数的值，如果是则条件为真。	(A >= B) 不为真。
<=	检查左操作数的值是否小于或等于右操作数的值，如果是则条件为真。	(A <= B) 为真。

逻辑运算符

下表显示了 C# 支持的所有逻辑运算符。假设变量 **A** 为布尔值 true，变量 **B** 为布尔值 false，则：

运算符	描述	实例
&&	称为逻辑与运算符。如果两个操作数都非零，则条件为真。	(A && B) 为假。
	称为逻辑或运算符。如果两个操作数中有任意一个非零，则条件为真。	(A B) 为真。
!	称为逻辑非运算符。用来逆转操作数的逻辑状态。如果条件为真则逻辑非运算符将使其为假。	!(A && B) 为真。

位运算符

位运算符作用于位，并逐位执行操作。&、| 和 ^ 的真值表如下所示：

p	q	p & q	p q	p ^ q
0	0	0	0	0
0	1	0	1	1
1	1	1	1	0
1	0	0	1	1

假设如果 A = 60，且 B = 13，现在以二进制格式表示，它们如下所示：

A = 0011 1100

B = 0000 1101

A&B = 0000 1100 全相同为一样，不一样为0，也就是11为1，00为0，01和10为0

A|B = 0011 1101 除了00为0，其他的都是1，也就是11为1，01为1，10为1

A^B = 0011 0001 除了11为0，其他的都是1，也就是00为1，01为1，10为1

~A = 1100 0011 反码，把01对换，也就是1变0，0变1

C# 支持的位运算符。假设变量 A 的值为 60，变量 B 的值为 13，则：

运算符	描述	实例
&	如果同时存在于两个操作数中，二进制 AND 运算符复制一位到结果中。	(A & B) 将得到 12，即为 0000 1100
	如果存在于任一操作数中，二进制 OR 运算符复制一位到结果中。	(A B) 将得到 61，即为 0011 1101
^	如果存在于其中一个操作数中但不同时存在于两个操作数中，二进制异或运算符复制一位到结果中。	(A ^ B) 将得到 49，即为 0011 0001
~	按位取反运算符是一元运算符，具有"翻转"位效果，即0变成1，1变成0，包括符号位。	(~A) 将得到 -61，即为 1100 0011，一个有符号制数的补码形式。
<<	二进制左移运算符。左操作数的值向左移动右操作数指定的位数。	A << 2 将得到 240，即为 1111 0000
>>	二进制右移运算符。左操作数的值向右移动右操作数指定的位数。	A >> 2 将得到 15，即为 0000 1111

赋值运算符

下表列出了 C# 支持的赋值运算符：

运算符	描述	实例
=	简单的赋值运算符，把右边操作数的值赋给左边操作数	C = A + B 将把 A + B 的值赋给 C
+=	加且赋值运算符，把右边操作数加上左边操作数的结果赋值给左边操作数	C += A 相当于 C = C + A
-=	减且赋值运算符，把左边操作数减去右边操作数的结果赋值给左边操作数	C -= A 相当于 C = C - A
*=	乘且赋值运算符，把右边操作数乘以左边操作数的结果赋值给左边操作数	C *= A 相当于 C = C * A
/=	除且赋值运算符，把左边操作数除以右边操作数的结果赋值给左边操作数	C /= A 相当于 C = C / A
%=	求模且赋值运算符，求两个操作数的模赋值给左边操作数	C %= A 相当于 C = C % A
<<=	左移且赋值运算符	C <<= 2 等同于 C = C << 2
>>=	右移且赋值运算符	C >>= 2 等同于 C = C >> 2
&=	按位与且赋值运算符	C &= 2 等同于 C = C & 2
^=	按位异或且赋值运算符	C ^= 2 等同于 C = C ^ 2
=	按位或且赋值运算符	C = 2 等同于 C = C 2

其他运算符

下表列出了 C# 支持的其他一些重要的运算符，包括 **sizeof**、**typeof** 和 **?:**。

运算符	描述	实例
sizeof()	返回数据类型的大小。	sizeof(int), 将返回 4.
typeof()	返回 class 的类型。	typeof(StreamReader);
&	返回变量的地址。	&a; 将得到变量的实际地址。
*	变量的指针。	*a; 将指向一个变量。
? :	条件表达式	如果条件为真 ? 则为 X : 否则为 Y
is	判断对象是否为某一类型。	If(Ford is Car) // 检查 Ford 是否是 Car 类的对象。
as	强制转换，即使转换失败也不会抛出异常。	Object obj = new StreamReader("Hello"); StreamReader r = obj as StreamReader;

C# 中的运算符优先级

优先级简易概括：有括号先括号，后乘除在加减，然后位移再关系，逻辑完后条件，最后一个逗号，。

类别	运算符	结合性
后缀	() [] -> . ++ --	从左到右
一元	+ - ! ~ ++ -- (type)* & sizeof	从右到左
乘除	* / %	从左到右
加减	+ -	从左到右
移位	<< >>	从左到右
关系	< <= > >=	从左到右
相等	== !=	从左到右
位与 AND	&	从左到右
位异或 XOR	^	从左到右
位或 OR		从左到右
逻辑与 AND	&&	从左到右
逻辑或 OR		从左到右
条件	?:	从右到左
赋值	= += -= *= /= %= >>= <<= &= ^= =	从右到左
逗号	,	从左到右

10.C#判断

判断语句

C# 提供了以下类型的判断语句。点击链接查看每个语句的细节。

语句	描述
if 语句	一个 if 语句 由一个布尔表达式后跟一个或多个语句组成。
if...else 语句	一个 if 语句 后可跟一个可选的 else 语句 , else 语句在布尔表达式为假时执行。
嵌套 if 语句	您可以在一个 if 或 else if 语句内使用另一个 if 或 else if 语句。
switch 语句	一个 switch 语句允许测试一个变量等于多个值时的情况。
嵌套 switch 语句	您可以在一个 switch 语句内使用另一个 switch 语句。

? : 运算符

我们已经在前面的章节中讲解了 **条件运算符 ? :**，可以用来替代 **if...else** 语句。它的一般形式如下：

```
1 Exp1 ? Exp2 : Exp3;
```

其中，Exp1、Exp2 和 Exp3 是表达式。请注意，冒号的使用和位置。
? 表达式的值是由 Exp1 决定的。如果 Exp1 为真，则计算 Exp2 的值，结果即为整个 ? 表达式的值。如果 Exp1 为假，则计算 Exp3 的值，结果即为整个 ? 表达式的值。

11.C#循环

循环类型

C# 提供了以下几种循环类型。点击链接查看每个类型的细节。

循环类型	描述
while 循环	当给定条件为真时，重复语句或语句组。它会在执行循环主体之前测试条
for/foreach 循环	多次执行一个语句序列，简化管理循环变量的代码。
do...while 循环	除了它是在循环主体结尾测试条件外，其他与 while 语句类似。
嵌套循环	您可以在 while、for 或 do..while 循环内使用一个或多个循环。

foreach

C# 也支持 foreach 循环，使用 foreach 可以迭代数组或者一个集合对象。
C# 的 foreach 循环可以用来遍历集合类型，例如数组、列表、字典等。它是一个简化版的 for 循环，使得代码更加简洁易读。
以下是 foreach 循环的语法：

```
1 foreach (var item in collection)
2 {
3     // 循环
4 }
```

collection 是要遍历的集合，item 是当前遍历到的元素。
以下实例有三个部分：

- 通过 foreach 循环输出整型数组中的元素。
- 通过 for 循环输出整型数组中的元素。
- foreach 循环设置数组元素的计算器。

C# 中 **嵌套 for 循环** 语句的语法：

```
1 for ( init; condition; increment )
2 {
3     for ( init; condition; increment )
4     {
5         statement(s);
6     }
7     statement(s);
8 }
```

C# 中 **嵌套 while 循环** 语句的语法：

```
1 while(condition)
2 {
3     while(condition)
4     {
5         statement(s);
6     }
7     statement(s);
8 }
```

C# 中 **嵌套 do...while 循环** 语句的语法:

```
1  do
2  {
3      statement(s);
4      do
5      {
6          statement(s);
7      }while( condition );
8
9  }while( condition );
```

关于嵌套循环有一点值得注意，您可以在任何类型的循环内嵌套其他任何类型的循环。比如，一个 for 循环可以嵌套在一个 while 循环内，反之亦然。

12.C#封装

封装 被定义为"把一个或多个项目封闭在一个物理的或者逻辑的包中"。在面向对象程序设计方法论中，封装是为了防止对实现细节的访问。

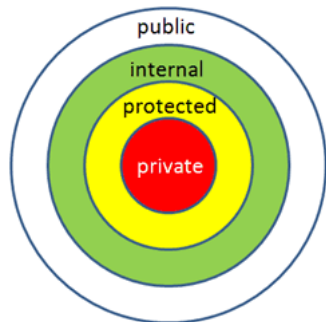
抽象和封装是面向对象程序设计的相关特性。

抽象允许相关信息可视化，封装则使开发者实现*所需级别的抽象*。

C# 封装根据具体的需要，设置使用者的访问权限，并通过 **访问修饰符** 来实现。

一个 **访问修饰符** 定义了一个类成员的范围和可见性。C# 支持的访问修饰符如下所示：

- public: 所有对象都可以访问；
- private: 对象本身在对象内部可以访问；
- protected: 只有该类对象及其子类对象可以访问
- internal: 同一个程序集的对象可以访问；
- protected internal: 访问限于当前程序集或派生自包含类的类型。



```
using System;
namespace RectangleApplication
{
    class Rectangle
    {
        // 私有成员变量
        private double length;
        private double width;
```

```
1  using System;
2
3  namespace RectangleApplication
4  {
5      class Rectangle
6      {
7          // 私有成员变量
8          private double length;
9          private double width;
10
11         // 公有方法，用于从用户输入获取矩形的长度和宽度
12         public void AcceptDetails()
13         {
14             Console.WriteLine("请输入长度: ");
```

```

15         length = Convert.ToDouble(Console.ReadLine());
16         Console.WriteLine("请输入宽度: ");
17         width = Convert.ToDouble(Console.ReadLine());
18     }
19
20     // 公有方法，用于计算矩形的面积
21     public double GetArea()
22     {
23         return length * width;
24     }
25
26     // 公有方法，用于显示矩形的属性和面积
27     public void Display()
28     {
29         Console.WriteLine("长度: {0}", length);
30         Console.WriteLine("宽度: {0}", width);
31         Console.WriteLine("面积: {0}", GetArea());
32     }
33 }//end class Rectangle
34
35 class ExecuteRectangle
36 {
37     static void Main(string[] args)
38     {
39         // 创建 Rectangle 类的实例
40         Rectangle r = new Rectangle();
41
42         // 通过公有方法 AcceptDetails() 从用户输入获取矩形的长度和宽度
43         r.AcceptDetails();
44
45         // 通过公有方法 Display() 显示矩形的属性和面积
46         r.Display();
47
48         Console.ReadLine();
49     }
50 }
51 }

```

说明：

- `length` 和 `width` 被声明为私有成员变量，以防止直接在类的外部访问和修改。
- `AcceptDetails` 方法允许用户输入矩形的长度和宽度，这是通过公有方法来操作私有变量的一个例子。
- `GetArea` 方法用于计算矩形的面积，而 `Display` 方法用于显示矩形的属性和面积。
- 在 `ExecuteRectangle` 类中，通过创建 `Rectangle` 类的实例，然后调用其公有方法，来执行操作。这样，主程序无法直接访问和修改矩形的长度和宽度，而是通过类提供的公有接口来进行操作，实现了封装。

13.C#方法

一个方法是把一些相关的语句组织在一起，用来执行一个任务的语句块。每一个 C# 程序至少有一个带有 Main 方法的类。

要使用一个方法，您需要：

- 定义方法
- 调用方法

C# 中定义方法

当定义一个方法时，从根本上说是在声明它的结构的元素。在 C# 中，定义方法的语法如下：

```

1  <Access Specifier> <Return Type> <Method Name>(<Parameter List>)
2  {
3      Method Body
4  }
5  例子

```

```
6  class NumberManipulator
7  {
8      public int FindMax(int num1, int num2)
9      {
10         /* 局部变量声明 */
11         int result;
12
13         if (num1 > num2)
14             result = num1;
15         else
16             result = num2;
17
18         return result;
19     }
20     ...
21 }
22
23 说明:
24 <Access Specifier> -> public
25 <Return Type> -> int
26 <Method Name> -> FindMax
27 (Parameter List) -> (int num1, int num2)
28 Method Body -> int result;
29
30         if (num1 > num2)
31             result = num1;
32         else
33             result = num2;
34
35         return result;
```

参数传递

当调用带有参数的方法时，您需要向方法传递参数。在 C# 中，有三种向方法传递参数的方式：

方式	描述
值参数	这种方式复制参数的实际值给函数的形式参数，实参和形参使用的是两个存中的值。在这种情况下，当形参的值发生改变时，不会影响实参的值，证了实参数据的安全。
引用参数	这种方式复制参数的内存位置的引用给形式参数。这意味着，当形参的值变时，同时也改变实参的值。
输出参数	这种方式可以返回多个值。

14.C#可空类型（Nullable）

C# 单问号 ? 与 双问号 ??

? 单问号用于对 int、double、bool 等无法直接赋值为 null 的数据类型进行 null 的赋值，意思是这个数据类型是 Nullable 类型的。

```
1  int? i = 3;
```

等同于：

```
1  Nullable<int> i = new Nullable<int>(3);
```

```
1  int i; //默认值0
```

```
2 int? ii; //默认值null
```

?? 双问号用于判断一个变量在为 null 的时候返回一个指定的值。

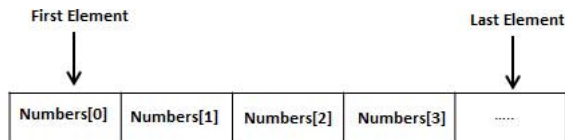
nullable 类型（可空类型）的语法如下：

```
1 < data_type> ? <variable_name> = null;
```

Null 合并运算符（ ?? ）

Null 合并运算符用于定义可空类型和引用类型的默认值。Null 合并运算符为类型转换定义了一个预设值，以防可空类型的值为 Null。Null 合并运算符把操作数类型隐式转换为另一个可空（或不可空）的值类型的操作数的类型。

15.C#数组（Array）



声明数组

在 C# 中声明一个数组，您可以使用下面的语法：

```
1 datatype[] arrayName;
```

其中，

- *datatype* 用于指定被存储在数组中的元素的类型。
- *[]* 指定数组的秩（维度）。秩指定数组的大小。
- *arrayName* 指定数组的名称。

例如：

```
1 double[] balance;
```

初始化数组

声明一个数组不会在内存中初始化数组。当初始化数组变量时，您可以赋值给数组。

数组是一个引用类型，所以您需要使用 **new** 关键字来创建数组的实例。

例如：

```
1 double[] balance = new double[10];
```

赋值给数组

```
1 可以通过使用索引号赋值给一个单独的数组元素，比如：
```

```
2 double[] balance = new double[10];
```

```
3 balance[0] = 4500.0;
```

```
4 可以在声明数组的同时给数组赋值，比如：
```

```
5 double[] balance = { 2340.0, 4523.69, 3421.0};
```

```
6 也可以创建并初始化一个数组，比如：
```

```
7 int [] marks = new int[5] { 99, 98, 92, 97, 95};
```

```
8 也可以省略数组的大小
```

```
9 int [] marks = new int[] { 99, 98, 92, 97, 95};
```

```
10 也可以赋值一个数组变量到另一个目标数组变量中。在这种情况下，目标和源会指向相同的内存位置：
```

```
11 int [] marks = new int[] { 99, 98, 92, 97, 95};
```

```
12  int[] score = marks;
```

C#多维数组

```
1  可以声明一个 string 变量的二维数组，如下：
2  string [,] names;
3  声明一个 int 变量的三维数组，如下：
4  int [ , , ] m;
```

C# 数组细节

在 C# 中，数组是非常重要的，且需要了解更多的细节。下面列出了 C# 程序员必须清楚的一些与数组相关的重要概念：

概念	描述
多维数组	C# 支持多维数组。多维数组最简单的形式是二维数组。
交错数组	C# 支持交错数组，即数组的数组。
传递数组给函数	您可以通过指定不带索引的数组名称来给函数传递一个指向数组的指针。
参数数组	这通常用于传递未知数量的参数给函数。
Array 类	在 System 命名空间中定义，是所有数组的基类，并提供了各种用于数组和方法。

二维数组

多维数组最简单的形式是二维数组。一个二维数组，在本质上，是一个一维数组的列表。
一个二维数组可以被认为是一个带有 x 行和 y 列的表格。下面是一个二维数组，包含 3 行和 4 列：

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

因此，数组中的每个元素是使用形式为 a[i,j] 的元素名称来标识的，其中 a 是数组名称，i 和 j 是唯一标识 a 中每个元素的下标。

初始化二维数组

多维数组可以通过在括号内为每行指定值来进行初始化。下面是一个带有 3 行 4 列的数组。

```
1  int [,] a = new int [3,4] {
2  {0, 1, 2, 3} ,      /* 初始化索引为 0 的行 */
3  {4, 5, 6, 7} ,      /* 初始化索引为 1 的行 */
4  {8, 9, 10, 11}      /* 初始化索引为 2 的行 */
5  };
```

C#交错数组

交错数组是数组的数组。
交错数组是一维数组。

```
1  可以声明一个带有 int 值的交错数组 scores，如下所示：
2  int [][] scores;
3
4  声明一个数组不会在内存中创建数组。创建上面的数组：
5  int[][] scores = new int[5][];
6  for (int i = 0; i < scores.Length; i++)
7  {
8      scores[i] = new int[4];
```

```

9  }
10
11 可以初始化一个交错数组，如下所示：
12  int[][] scores = new int[2][]{new int[]{92,93,94},new int[]{85,66,87,88}};
13  scores 是一个由两个整型数组组成的数组 -- scores[0] 是一个带有 3 个整数的数组，scores[1] 是一个带有 4 个整数的数组。
14
15 如何使用交错数组：
16  using System;
17
18  namespace ArrayApplication
19  {
20      class MyArray
21      {
22          static void Main(string[] args)
23          {
24              /* 一个由 5 个整型数组组成的交错数组 */
25              int[][] a = new int[][]{new int[]{0,0},new int[]{1,2},
26              new int[]{2,4},new int[]{ 3, 6 }, new int[]{ 4, 8 } };
27
28              int i, j;
29
30              /* 输出数组中每个元素的值 */
31              for (i = 0; i < 5; i++)
32              {
33                  for (j = 0; j < 2; j++)
34                  {
35                      Console.WriteLine("a[{0}][{1}] = {2}", i, j, a[i][j]);
36                  }
37              }
38              Console.ReadKey();
39          }
40      }
41  }

```

C#参数数组

当声明一个方法时，您不能确定要传递给函数作为参数的参数数目。C# 参数数组解决了这个问题，参数数组通常用于传递未知数量的参数给函数。

Params关键字

在使用数组作为形参时，C# 提供了 params 关键字，使调用数组为形参的方法时，既可以传递数组实参，也可以传递一组数组元素。params 的使用格式为：

```

1  public 返回类型 方法名称( params 类型名称[] 数组名称 )
2
3  实例：
4  using System;
5
6  namespace ArrayApplication
7  {
8      class ParamArray
9      {
10         public int AddElements(params int[] arr)
11         {
12             int sum = 0;
13             foreach (int i in arr)
14             {
15                 sum += i;
16             }
17             return sum;
18         }
19     }
20
21     class TestClass
22     {

```



```
23     static void Main(string[] args)
24     {
25         ParamArray app = new ParamArray();
26         int sum = app.AddElements(512, 720, 250, 567, 889);
27         Console.WriteLine("总和是: {0}", sum);
28         Console.ReadKey();
29     }
30 }
31 }
32
33 结果:
34 总和是: 2938
```

16.C#字符串（String）

创建 String 对象

可以使用以下方法之一来创建 string 对象：

- 通过给 String 变量指定一个字符串
- 通过使用 String 类构造函数
- 通过使用字符串串联运算符（+）
- 通过检索属性或调用一个返回字符串的方法
- 通过格式化方法来转换一个值或对象为它的字符串表示形式

String 类的属性

String 类有以下两个属性：

序号	属性名称 & 描述
1	Chars 在当前 <i>String</i> 对象中获取 <i>Char</i> 对象的指定位置。
2	Length 在当前的 <i>String</i> 对象中获取字符数。

String 类的方法

String 类有许多方法用于 string 对象的操作。下面的表格提供了一些最常用的方法：

序号	方法名称 & 描述
1	public static int Compare(string strA, string strB) 比较两个指定的 string 对象，并返回一个表示它们在排列顺序中相对位置的整数。该方法区分大小写。
2	public static int Compare(string strA, string strB, bool ignoreCase) 比较两个指定的 string 对象，并返回一个表示它们在排列顺序中相对位置的整数。但是，如果布尔参数为真时，该方法不区分大小写。
3	public static string Concat(string str0, string str1) 连接两个 string 对象。
4	public static string Concat(string str0, string str1, string str2) 连接三个 string 对象。
5	public static string Concat(string str0, string str1, string str2, string str3) 连接四个 string 对象。
6	public bool Contains(string value) 返回一个表示指定 string 对象是否出现在字符串中的值。
7	public static string Copy(string str) 创建一个与指定字符串具有相同值的新的 String 对象。
8	public void CopyTo(int sourceIndex, char[] destination, int destinationIndex, int count) 从 string 对象的指定位置开始复制指定数量的字符到 Unicode 字符数组中的指定位置。
9	public bool EndsWith(string value) 判断 string 对象的结尾是否匹配指定的字符串。
10	public bool Equals(string value) 判断当前的 string 对象是否与指定的 string 对象具有相同的值。
11	public static bool Equals(string a, string b) 判断两个指定的 string 对象是否具有相同的值。
12	public static string Format(string format, Object arg0) 把指定字符串中一个或多个格式项替换为指定对象的字符串表示形式。
13	public int IndexOf(char value) 返回指定 Unicode 字符在当前字符串中第一次出现的索引，索引从 0 开始。
14	public int IndexOf(string value) 返回指定字符串在该实例中第一次出现的索引，索引从 0 开始。
15	public int IndexOf(char value, int startIndex) 返回指定 Unicode 字符从该字符串中指定字符位置开始搜索第一次出现的索引，索引从 0 开始。
16	public int IndexOf(string value, int startIndex) 返回指定字符串从该实例中指定字符位置开始搜索第一次出现的索引，索引从 0 开始。
17	public int IndexOfAny(char[] anyOf) 返回某一个指定的 Unicode 字符数组中任意字符在该实例中第一次出现的索引，索引从 0 开始。
18	public int IndexOfAny(char[] anyOf, int startIndex) 返回某一个指定的 Unicode 字符数组中任意字符从该实例中指定字符位置开始搜索第一次出现的索引，索引从 0 开始。
19	public string Insert(int startIndex, string value) 返回一个新的字符串，其中，指定的字符串被插入在当前 string 对象的指定索引位置。
20	public static bool IsNullOrEmpty(string value) 指示指定的字符串是否为 null 或者是否为一个空的字符串。
21	public static string Join(string separator, string[] value) 连接一个字符串数组中的所有元素，使用指定的分隔符分隔每个元素。
22	public static string Join(string separator, string[] value, int startIndex, int count) 连接一个字符串数组中的指定位置开始的指定元素，使用指定的分隔符分隔每个元素。
23	public int LastIndexOf(char value)

	返回指定 Unicode 字符在当前 string 对象中最后一次出现的索引位置，索引从 0 开始。
24	public int LastIndexOf(string value) 返回指定字符串在当前 string 对象中最后一次出现的索引位置，索引从 0 开始。
25	public string Remove(int startIndex) 移除当前实例中的所有字符，从指定位置开始，一直到最后一个位置为止，并返回字符串。
26	public string Remove(int startIndex, int count) 从当前字符串的指定位置开始移除指定数量的字符，并返回字符串。
27	public string Replace(char oldChar, char newChar) 把当前 string 对象中，所有指定的 Unicode 字符替换为另一个指定的 Unicode 字符，并返回新的字符串。
28	public string Replace(string oldValue, string newValue) 把当前 string 对象中，所有指定的字符串替换为另一个指定的字符串，并返回新的字符串。
29	public string[] Split(params char[] separator) 返回一个字符串数组，包含当前的 string 对象中的子字符串，子字符串是使用指定的 Unicode 字符数组中的元素进行分隔的。
30	public string[] Split(char[] separator, int count) 返回一个字符串数组，包含当前的 string 对象中的子字符串，子字符串是使用指定的 Unicode 字符数组中的元素进行分隔的。int 参数指定要返回的子字符串最大数目。
31	public bool StartsWith(string value) 判断字符串实例的开头是否匹配指定的字符串。
32	public char[] ToCharArray() 返回一个带有当前 string 对象中所有字符的 Unicode 字符数组。
33	public char[] ToCharArray(int startIndex, int length) 返回一个带有当前 string 对象中所有字符的 Unicode 字符数组，从指定的索引开始，直到指定的长度为止。
34	public string ToLower() 把字符串转换为小写并返回。
35	public string ToUpper() 把字符串转换为大写并返回。
36	public string Trim() 移除当前 String 对象中的所有前导空白字符和后置空白字符。

17.C#结构体（Struct）

在 C# 中，结构体（struct）是一种值类型（value type），用于组织和存储相关数据。
在 C# 中，结构体是值类型数据结构，这样使得一个单一变量可以存储各种数据类型的相关数据。
struct 关键字用于创建结构体。

结构体是用来代表一个记录，假设您想跟踪图书馆中书的动态，您可能想跟踪每本书的以下属性：

- Title
- Author
- Subject
- Book ID

定义结构体

为了定义一个结构体，您必须使用 **struct** 语句。
struct 语句为程序定义了一个带有多个成员的新的数据类型。
例如，您可以按照如下的方式声明 Book 结构：

```
1 struct Books
2 {
3     public string title;
4     public string author;
5     public string subject;
```

```
6     public int book_id;
7 };
```

C# 结构的特点

结构提供了一种轻量级的数据类型，适用于表示简单的数据结构，具有较好的性能特性和值语义：

- 结构可带有方法、字段、索引、属性、运算符方法和事件，适用于表示轻量级数据的情况，如坐标、范围、日期、时间等。
- 结构可**定义构造函数，但不能定义析构函数**。但是，不能为结构定义无参构造函数。无参构造函数(默认)是自动定义的，且不能被改变。
- 与类不同，**结构不能继承其他的结构或类**。
- 结构不能作为其他结构或类的基础结构。
- 结构可实现**一个或多个**接口。
- 结构**成员不能指定为 abstract、virtual 或 protected**。
- 使用 **New** 操作符创建一个结构对象时，会调用适当的构造函数来创建结构。与类不同，**结构可以不使用 New 操作符即可被实例化**。
- 如果不使用 New 操作符，只有在所有的字段都被初始化之后，字段才被赋值，对象才被使用。
- 结构变量通常分配在栈上，这使得它们的创建和销毁速度更快。但是，如果将结构用作类的字段，且这个类是引用类型，那么结构将存储在堆上。
- 结构默认情况下是可变的，这意味着可以修改它们的字段。但是，如果结构定义为只读，那么它的字段将是不可变的

类 vs 结构

值类型 vs 引用类型：

- **结构是值类型 (Value Type)：** 结构是值类型，它们在栈上分配内存，而不是在堆上。当将结构实例传递给方法或赋值给另一个变量时，将复制整个结构的内容。
- **类是引用类型 (Reference Type)：** 类是引用类型，它们在堆上分配内存。当将类实例传递给方法或赋值给另一个变量时，实际上是传递引用（内存地址）而不是整个对象的副本。

继承和多态性：

- **结构不能继承：** 结构不能继承其他结构或类，也不能作为其他结构或类的基类。
- **类支持继承：** 类支持继承和多态性，可以通过派生新类来扩展现有类的功能。

默认构造函数：

- **结构不能有无参数的构造函数：** 结构不能包含无参数的构造函数。
- **类可以有参数的构造函数：** 类可以包含无参数的构造函数，如果没有提供构造函数，系统会提供默认的无参数构造函数。

赋值行为：

- 类型为类的变量在赋值时存储的是引用，因此两个变量指向同一个对象。
- 结构变量在赋值时会复制整个结构，因此每个变量都有自己的独立副本。

传递方式：

- 类型为类的对象在方法调用时通过引用传递，这意味着在方法中对对象所做的更改会影响到原始对象。
- 结构对象通常通过值传递，这意味着传递的是结构的副本，而不是原始结构对象本身。因此，在方法中对结构所做的更改不会影响到原始对象。

可空性：

- **结构体是值类型，不能直接设置为 null：** 因为 **null** 是引用类型的默认值，而不是值类型的默认值。如果你需要表示结构体变量的缺失或无效状态，可以使用 **Nullable<T>** 或称为 **T?** 的可空类型。
- **类默认可为null：** 类的实例默认可以为 **null**，因为它们是引用类型。

性能和内存分配：

- **结构通常更轻量：** 由于结构是值类型且在栈上分配内存，它们通常比类更轻量，适用于简单的数据表示。
- **类可能有更多开销：** 由于类是引用类型，可能涉及更多的内存开销和管理。

```
1 using System;
2
3 // 结构声明
4 struct MyStruct
5 {
6     public int X;
7     public int Y;
8
9     // 结构不能有无参数的构造函数
10    // public MyStruct()
11    // {
12    // }
13
14    // 有参数的构造函数
15    public MyStruct(int x, int y)
16    {
17        X = x;
```

```
18     Y = y;
19 }
20
21 // 结构不能继承
22 // struct MyDerivedStruct : MyBaseStruct
23 // {
24 // }
25 }
26
27 // 类声明
28 class MyClass
29 {
30     public int X;
31     public int Y;
32
33     // 类可以有参数的构造函数
34     public MyClass()
35     {
36     }
37
38     // 有参数的构造函数
39     public MyClass(int x, int y)
40     {
41         X = x;
42         Y = y;
43     }
44
45     // 类支持继承
46     // class MyDerivedClass : MyBaseClass
47     // {
48     // }
49 }
50
51 class Program
52 {
53     static void Main()
54     {
55         // 结构是值类型，分配在栈上
56         MyStruct structInstance1 = new MyStruct(1, 2);
57         MyStruct structInstance2 = structInstance1; // 复制整个结构
58
59         // 类是引用类型，分配在堆上
60         MyClass classInstance1 = new MyClass(3, 4);
61         MyClass classInstance2 = classInstance1; // 复制引用，指向同一个对象
62
63         // 修改结构实例不影响其他实例
64         structInstance1.X = 5;
65         Console.WriteLine($"Struct: {structInstance1.X}, {structInstance2.X}");
66
67         // 修改类实例会影响其他实例
68         classInstance1.X = 6;
69         Console.WriteLine($"Class: {classInstance1.X}, {classInstance2.X}");
70     }
71 }
72
73
74 例子:
75 using System;
76 using System.Text;
77
78 struct Books
79 {
80     private string title;
```

```

81     private string author;
82     private string subject;
83     private int book_id;
84     public void setValues(string t, string a, string s, int id)
85     {
86         title = t;
87         author = a;
88         subject = s;
89         book_id =id;
90     }
91     public void display()
92     {
93         Console.WriteLine("Title : {0}", title);
94         Console.WriteLine("Author : {0}", author);
95         Console.WriteLine("Subject : {0}", subject);
96         Console.WriteLine("Book_id :{0}", book_id);
97     }
98
99 };
100
101 public class testStructure
102 {
103     public static void Main(string[] args)
104     {
105
106         Books Book1 = new Books(); /* 声明 Book1, 类型为 Books */
107         Books Book2 = new Books(); /* 声明 Book2, 类型为 Books */
108
109         /* book 1 详述 */
110         Book1.setValues("C Programming",
111             "Nuha Ali", "C Programming Tutorial",6495407);
112
113         /* book 2 详述 */
114         Book2.setValues("Telecom Billing",
115             "Zara Ali", "Telecom Billing Tutorial", 6495700);
116
117         /* 打印 Book1 信息 */
118         Book1.display();
119
120         /* 打印 Book2 信息 */
121         Book2.display();
122
123         Console.ReadKey();
124
125     }
126 }
127
128 Title : C Programming
129 Author : Nuha Ali
130 Subject : C Programming Tutorial
131 Book_id : 6495407
132 Title : Telecom Billing
133 Author : Zara Ali
134 Subject : Telecom Billing Tutorial
135 Book_id : 6495700

```

18.C# 枚举 (Enum)

枚举是一组命名整型常量。枚举类型是使用 **enum** 关键字声明的。

C# 枚举是值类型。换句话说，枚举包含自己的值，且不能继承或传递继承。

声明 **enum** 变量

声明枚举的一般语法：

```
1  enum <enum_name>
2  {
3      enumeration list
4  };
5
6  其中，
7  enum_name 指定枚举的类型名称。
8  enumeration list 是一个用逗号分隔的标识符列表。
9  枚举列表中的每个符号代表一个整数值，一个比它前面的符号大的整数值。默认情况下，第一个枚举符号的值是 0。例如：
10 enum Days { Sun, Mon, tue, Wed, thu, Fri, Sat };
11
12
13 例子：
14 using System;
15
16 public class EnumTest
17 {
18     enum Day { Sun, Mon, Tue, Wed, Thu, Fri, Sat };
19
20     static void Main()
21     {
22         int x = (int)Day.Sun;
23         int y = (int)Day.Fri;
24         Console.WriteLine("Sun = {0}", x);
25         Console.WriteLine("Fri = {0}", y);
26     }
27 }
```

19.C#继承

当创建一个类时，程序员不需要完全重新编写新的数据成员和成员函数，只需要设计一个新的类，继承了已有的类的成员即可。这个已有的类被称为的**基类**，这个新的类被称为**派生类**。

继承的思想实现了 **属于 (IS-A)** 关系。例如，哺乳动物 **属于 (IS-A)** 动物，狗 **属于 (IS-A)** 哺乳动物，因此狗 **属于 (IS-A)** 动物。

基类和派生类

一个类可以继承自另一个类，被称为基类（父类）和派生类（子类）。
C# 不支持类的多重继承，但支持接口的多重继承，一个类可以实现多个接口。

概括来说：一个类可以继承多个接口，但只能继承自一个类。

C# 中创建派生类的语法如下：

```
<访问修饰符> class <基类>
{
    ...
}
class <派生类> : <基类>
{
    ...
}
```

派生类会继承基类的成员（字段、方法、属性等），除非它们被明确地标记为私有（private）。
派生类可以通过关键字base来调用基类的构造函数和方法。

继承接口（Interface Inheritance）

一个接口可以继承自一个或多个其他接口，派生接口继承了基接口的所有成员。
派生接口可以扩展基接口的成员列表，但不能改变它们的访问修饰符。

实例

```
interface IBaseInterface
{
```

```
void Method1();  
}
```

```
interface IDerivedInterface : IBaseInterface  
{  
    void Method2();  
}
```

```
1  using System;  
2  
3  // 定义一个基接口  
4  interface IBaseInterface  
5  {  
6      void Method1();  
7  }  
8  
9  // 定义一个派生接口，继承自基接口  
10 interface IDerivedInterface : IBaseInterface  
11 {  
12     void Method2();  
13 }  
14  
15 // 实现派生接口的类  
16 class MyClass : IDerivedInterface  
17 {  
18     public void Method1()  
19     {  
20         Console.WriteLine("Method1 implementation");  
21     }  
22  
23     public void Method2()  
24     {  
25         Console.WriteLine("Method2 implementation");  
26     }  
27 }  
28  
29 class Program  
30 {  
31     static void Main(string[] args)  
32     {  
33         // 创建 MyClass 类的实例  
34         MyClass obj = new MyClass();  
35  
36         // 调用继承自基接口的方法  
37         obj.Method1();  
38  
39         // 调用派生接口新增的方法  
40         obj.Method2();  
41     }  
42 }  
43  
44 结果;  
45 Method1 implementation  
46 Method2 implementation
```

20.C#多态性

多态是同一个行为具有多个不同表现形式或形态的能力。

多态性意味着有多重形式。在面向对象编程范式中，多态性往往表现为“一个接口，多个功能”。

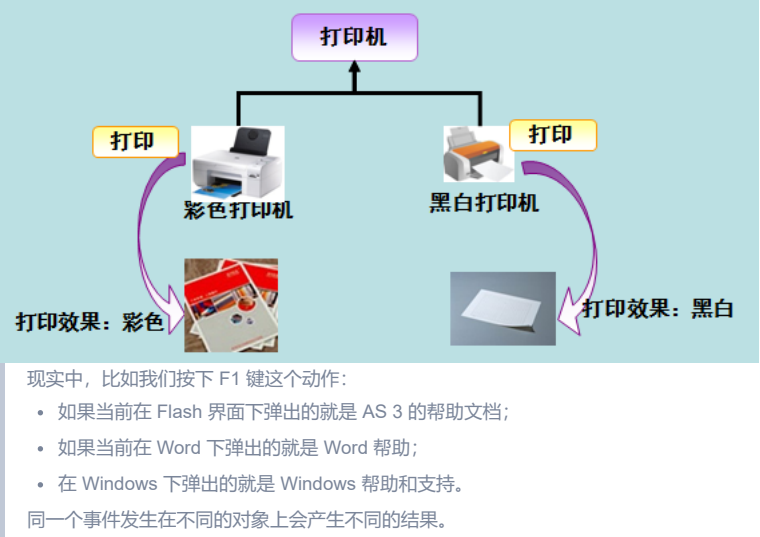
多态性可以是静态的或动态的。

在**静态多态性**中，函数的响应是在编译时发生的。

在**动态多态性**中，函数的响应是在运行时发生的。

在 C# 中，每个类型都是多态的，因为包括用户定义类型在内的所有类型都继承自 Object。

多态就是同一个接口，使用不同的实例而执行不同操作，如图所示：



静态多态性

在编译时，函数和对象的连接机制被称为早期绑定，也被称为静态绑定。C# 提供了两种技术来实现静态多态性。分别为：

- 函数重载
- 运算符重载

动态多态性

C# 允许使用关键字 **abstract** 创建抽象类，用于提供接口的部分类的实现。当一个派生类继承自该抽象类时，实现即完成。**抽象类**包含抽象方法，抽象方法可被派生类实现。派生类具有更专业的功能。

请注意，下面是有关抽象类的一些规则：

- 不能创建一个抽象类的实例。
- 不能在一个抽象类外部声明一个抽象方法。
- 通过在类定义前面放置关键字 **sealed**，可以将类声明为**密封类**。当一个类被声明为 **sealed** 时，它不能被继承。抽象类不能被声明为 sealed。

C# 允许使用关键字 **abstract** 创建抽象类，用于提供接口的部分类的实现。当一个派生类继承自该抽象类时，实现即完成。**抽象类**包含抽象方法，抽象方法可被派生类实现。派生类具有更专业的功能。

请注意，下面是有关抽象类的一些规则：

- 不能创建一个抽象类的实例。
- 不能在一个抽象类外部声明一个抽象方法。
- 通过在类定义前面放置关键字 **sealed**，可以将类声明为**密封类**。当一个类被声明为 **sealed** 时，它不能被继承。抽象类不能被声明为 sealed。

当有一个定义在类中的函数需要在继承类中实现时，可以使用**虚方法**。

虚方法是使用关键字 **virtual** 声明的。

虚方法可以在不同的继承类中有不同的实现。

对虚方法的调用是在运行时发生的。

动态多态性是通过 **抽象类** 和 **虚方法** 实现的。

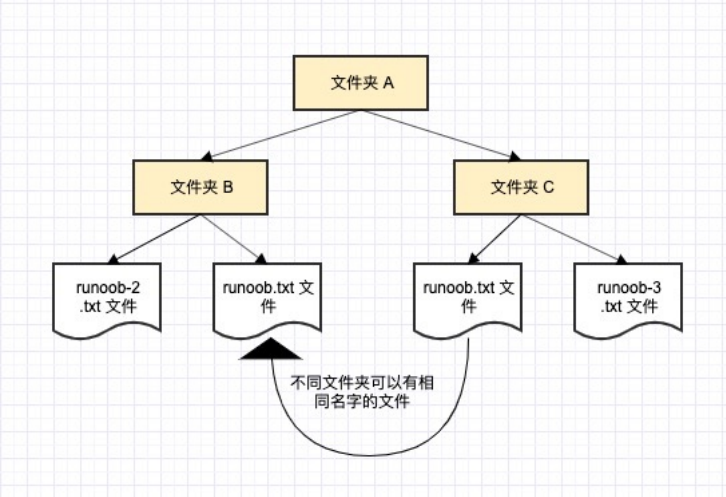
可重载和不可重载运算符

下表描述了 C# 中运算符重载的能力：

运算符	描述
+, -, !, ~, ++, --	这些一元运算符只有一个操作数，且可以被重载。
+, -, *, /, %	这些二元运算符带有两个操作数，且可以被重载。
==, !=, <, >, <=, >=	这些比较运算符可以被重载。
&&,	这些条件逻辑运算符 不能被直接重载 。
+=, -=, *=, /=, %=	这些赋值运算符 不能被重载 。
=, ., ?., ->, new, is, sizeof, typeof	这些运算符 不能被重载 。

21.C#命名空间

命名空间的设计目的是提供一种让一组名称与其他名称分隔开的方式。在一个命名空间中声明的类的名称与另一个命名空间中声明的相同的类的名称不冲突。举一个计算机系统中的一个例子，一个文件夹(目录)中可以包含多个文件夹，每个文件夹中不能有相同的文件名，但不同文件夹中的文件可以重名。



定义命名空间

```
1 命名空间的定义是以关键字 namespace 开始，后跟命名空间的名称，如下所示：
2 namespace namespace_name
3 {
4     // 代码声明
5 }
6
7 为了调用支持命名空间版本的函数或变量，会把命名空间的名称置于前面，如下所示：
8 namespace_name.item_name;
```

using 关键字

using 关键字表明程序使用的是给定命名空间中的名称。例如，我们在程序中使用 **System** 命名空间，其中定义了类 **Console**。我们可以只写：

```
1 Console.WriteLine ("Hello there");
```

我们可以写完全限定名称，如下：

```
1 System.Console.WriteLine("Hello there");
```

22.C#预处理器指令

预处理器指令（Preprocessor Directives）指导编译器在实际编译开始之前对信息进行预处理。通过这些指令，可以控制编译器如何编译文件或编译哪些部分。常见的预处理器指令包括条件编译、宏定义等。所有的预处理器指令都是以 **#** 开始，且在一行上，只有空白字符可以出现在预处理器指令之前。预处理器指令不是语句，所以它们不以分号 **;** 结束。C# 编译器没有一个单独的预处理器，但是，指令被处理时就像是有一个单独的预处理器一样。在 C# 中，预处理器指令用于在条件编译中起作用。与 C 和 C++ 不同的是，它们不是用来创建宏。一个预处理器指令必须是该行上的唯一指令。

C# 预处理器指令列表

下表列出了 C# 中可用的预处理器指令：

指令	描述
<code>#define</code>	定义一个符号，可以用于条件编译。
<code>#undef</code>	取消定义一个符号。
<code>#if</code>	开始一个条件编译块，如果符号被定义则包含代码块。
<code>#elif</code>	如果前面的 <code>#if</code> 或 <code>#elif</code> 条件不满足，且当前条件满足，则包含代码块。
<code>#else</code>	如果前面的 <code>#if</code> 或 <code>#elif</code> 条件不满足，则包含代码块。
<code>#endif</code>	结束一个条件编译块。
<code>#warning</code>	生成编译器警告信息。
<code>#error</code>	生成编译器错误信息。
<code>#region</code>	标记一段代码区域，可以在IDE中折叠和展开这段代码，便于代码的组织。
<code>#endregion</code>	结束一个代码区域。
<code>#line</code>	更改编译器输出中的行号和文件名，可以用于调试或生成工具的代码。
<code>#pragma</code>	用于给编译器发送特殊指令，例如禁用或恢复特定的警告。
<code>nullable</code>	控制可空性上下文和注释，允许启用或禁用对可空引用类型的编译器检查。

#define和#undef预处理器

#define用于定义符号（条件编译）

#undef用于取消定义符号

例子：

```
#define DEBUG
```

```
#undef RELEASE
```

条件指令：#if, #elif, #else 和 #endif

可以使用 `#if` 指令来创建一个条件指令。

条件指令用于测试符号是否为真。如果为真，编译器会执行 `#if` 和下一个指令之间的代码。

条件指令的语法：

```
1  #if symbol [operator symbol]...
```

symbol 是要测试的符号名称。可以使用 true 和 false，或在符号前放置否定运算符。

常见运算符有：

- == (等于)
- != (不等于)
- && (与)
- || (或)

也可以用括号把符号和运算符进行分组。条件指令用于在调试版本或编译指定配置时编译代码。一个以 `#if` 指令开始的条件指令，必须显示地以一个 `#endif` 指令终止。

```
1  #define DEBUG
2
3  #if DEBUG
4      Console.WriteLine("Debug mode");
5  #elif RELEASE
6      Console.WriteLine("Release mode");
7  #else
8      Console.WriteLine("Other mode");
```

```
9 #endif
```

#warning 和 #error

#warning 用于生成编译器警告，#error 用于生成编译器错误。

```
1 #warning This is a warning message
2 #error This is an error message
```

#region 和 #endregion

用于代码折叠，使代码更加可读。

```
1 #region MyRegion
2     // Your code here
3 #endregion
```

#line

用于更改文件行号和文件名的编译器输出。

```
1 #line 100 "MyFile.cs"
2     // The next line will be reported as line 100 in MyFile.cs
3     Console.WriteLine("This is line 100");
4 #line default
5     // Line numbering returns to normal
```

#pragma

用于向编译器发送特殊指令。最常见的用法是禁用特定的警告。

```
1 #pragma warning disable 414
2     private int unusedVariable;
3 #pragma warning restore 414
```

使用预处理器指令的注意事项

- **提高代码可读性**：使用 `#region` 可以帮助分隔代码块，提高代码的组织性。
- **条件编译**：通过 `#if` 等指令可以在开发和生产环境中编译不同的代码，方便调试和发布。
- **警告和错误**：通过 `#warning` 和 `#error` 可以在编译时提示开发人员注意特定问题。

通过正确使用这些预处理器指令，可以更好地控制代码的编译过程，提高代码的灵活性和可维护性。

23.C#正则表达式

正则表达式 是一种匹配输入文本的模式。

.Net 框架提供了允许这种匹配的正则表达式引擎。

模式由一个或多个字符、运算符和结构组成。

定义正则表达式

下面列出了用于定义正则表达式的各种类别的字符、运算符和结构。

- 字符转义
- 字符类
- 定位点
- 分组构造
- 限定符
- 反向引用构造

- 备用构造
- 替换
- 杂项构造

列表链接：[C# 正则表达式 | 菜鸟教程](#)

C#进阶

24.C#异常处理

异常提供了一种把程序控制权从某个部分转移到另一个部分的方式。C# 异常处理时建立在四个关键词之上的：**try**、**catch**、**finally** 和 **throw**。

- **try**：一个 try 块标识了一个将被激活的特定的异常的代码块。后跟一个或多个 catch 块。
- **catch**：程序通过异常处理程序捕获异常。catch 关键字表示异常的捕获。
- **finally**：finally 块用于执行给定的语句，不管异常是否被抛出都会执行。例如，如果您打开一个文件，不管是否出现异常文件都要被关闭。
- **throw**：当问题出现时，程序抛出一个异常。使用 throw 关键字来完成。

语法

假设一个块将出现异常，一个方法使用 try 和 catch 关键字捕获异常。try/catch 块内的代码为受保护的代码，使用 try/catch 语法如下所示：

```
1  try
2  {
3  // 引起异常的语句
4  }
5  catch( ExceptionName e1 )
6  {
7  // 错误处理代码
8  }
9  catch( ExceptionName e2 )
10 {
11 // 错误处理代码
12 }
13 catch( ExceptionName eN )
14 {
15 // 错误处理代码
16 }
17 finally
18 {
19 // 要执行的语句
20 }
```

可以列出多个 catch 语句捕获不同类型的异常，以防 try 块在不同的情况下生成多个异常

C# 中的异常类

C# 异常是使用类来表示的。C# 中的异常类主要是直接或间接地派生于 **System.Exception** 类。**System.ApplicationException** 和 **System.SystemException** 类是派生于 System.Exception 类的异常类。

System.ApplicationException 类支持由应用程序生成的异常。所以程序员定义的异常都应派生自该类。

System.SystemException 类是所有预定义的系统异常的基类

System.SystemException 类的预定义的异常类：

异常类	描述
System.IO.IOException	处理 I/O 错误。
System.IndexOutOfRangeException	处理当方法指向超出范围的数组索引时生成的错误。
System.ArrayTypeMismatchException	处理当数组类型不匹配时生成的错误。
System.NullReferenceException	处理当依从一个空对象时生成的错误。
System.DivideByZeroException	处理当除以零时生成的错误。
System.InvalidCastException	处理在类型转换期间生成的错误。
System.OutOfMemoryException	处理空闲内存不足生成的错误。
System.StackOverflowException	处理栈溢出生成的错误。

抛出对象

如果异常是直接或间接派生自 **System.Exception** 类，您可以抛出一个对象。您可以在 `catch` 块中使用 `throw` 语句来抛出当前的对象，如下所示：

```
1  Catch(Exception e)
2  {
3      ...
4      Throw e
5  }
```

25.C#文件的输入和输出

一个 **文件** 是一个存储在磁盘中带有指定名称和目录路径的数据集合。当打开文件进行读写时，它变成一个 **流**。

从根本上说，流是通过通信路径传递的字节序列。有两个主要的流：**输入流** 和 **输出流**。**输入流**用于从文件读取数据（读操作），**输出流**用于向文件写入数据（写操作）。

C# I/O 类

System.IO 命名空间有各种不同的类，**用于执行各种文件操作，如创建和删除文件、读取或写入文件，关闭文件等。**

下表列出了一些 System.IO 命名空间中常用的非抽象类：

I/O 类	描述
BinaryReader	从二进制流读取原始数据。
BinaryWriter	以二进制格式写入原始数据。
BufferedStream	字节流的临时存储。
Directory	有助于操作目录结构。
DirectoryInfo	用于对目录执行操作。
DriveInfo	提供驱动器的信息。
File	有助于处理文件。
FileInfo	用于对文件执行操作。
FileStream	用于文件中任何位置的读写。
MemoryStream	用于随机访问存储在内存中的数据流。
Path	对路径信息执行操作。
StreamReader	用于从字节流中读取字符。
StreamWriter	用于向一个流中写入字符。
StringReader	用于读取字符串缓冲区。
StringWriter	用于写入字符串缓冲区。

FileStream 类

System.IO 命名空间中的 **FileStream** 类有助于文件的读写与关闭。该类派生自抽象类 Stream。

您需要创建一个 **FileStream** 对象来创建一个新的文件，或打开一个已有的文件。创建 **FileStream** 对象的语法如下：

```
1 FileStream <object_name> = new FileStream( <file_name>,
2 <FileMode Enumerator>, <FileAccess Enumerator>, <FileShare Enumerator>);
3 例子:
4 FileStream F = new FileStream("sample.txt", FileMode.Open, FileAccess.Read, FileShare.Read);
```

例如，创建一个 FileStream 对象 **F** 来读取名为 **sample.txt** 的文件：

```
1 FileStream F = new FileStream("sample.txt", FileMode.Open, FileAccess.Read, FileShare.Read);
```

FileMode	FileMode 枚举定义了各种打开文件的方法。FileMode 枚举的成员有： • Append：打开一个已有的文件，并将光标放置在文件的末尾。如果文件不存在在文件。 • Create：创建一个新的文件。如果文件已存在，则删除旧文件，然后创建新文件。 • CreateNew：指定操作系统应创建一个新的文件。如果文件已存在，则抛出异常。 • Open：打开一个已有的文件。如果文件不存在，则抛出异常。 • OpenOrCreate：指定操作系统应打开一个已有的文件。如果文件不存在，则用称创建一个新的文件打开。 • Truncate：打开一个已有的文件，文件一旦打开，就将被截断为零字节大小。可以向文件写入全新的数据，但是保留文件的初始创建日期。如果文件不存在，异常。
FileAccess	FileAccess 枚举的成员有：Read、ReadWrite 和 Write。
FileShare	FileShare 枚举的成员有： • Inheritable：允许文件句柄可由子进程继承。Win32 不直接支持此功能。 • None：谢绝共享当前文件。文件关闭前，打开该文件的任何请求（由此进程或发出的请求）都将失败。 • Read：允许随后打开文件读取。如果未指定此标志，则文件关闭前，任何打开进行读取的请求（由此进程或另一进程发出的请求）都将失败。但是，即使指定志，仍可能需要附加权限才能够访问该文件。 • ReadWrite：允许随后打开文件读取或写入。如果未指定此标志，则文件关闭前开该文件以进行读取或写入的请求（由此进程或另一进程发出）都将失败。但指定了此标志，仍可能需要附加权限才能够访问该文件。 • Write：允许随后打开文件写入。如果未指定此标志，则文件关闭前，任何打开进行写入的请求（由此进程或另一进过程发出的请求）都将失败。但是，即使指定志，仍可能需要附加权限才能够访问该文件。 • Delete：允许随后删除文件。

C# 高级文件操作

主题	描述
文本文件的读写	它涉及到文本文件的读写。 StreamReader 和 StreamWriter 类有助于完成文本文件的读写。
二进制文件的读写	它涉及到二进制文件的读写。 BinaryReader 和 BinaryWriter 类有助于完成二进制文件的读写。
Windows 文件系统的操作	它让 C# 程序员能够浏览并定位 Windows 文件和目

文本文件的读写

StreamReader 和 **StreamWriter** 类用于文本文件的数据读写。这些类从抽象基类 **Stream** 继承，**Stream** 支持文件流的字节读写。

StreamReader 类

StreamReader 类继承自抽象基类 **TextReader**，表示阅读器读取一系列字符。

下表列出了 **StreamReader** 类中一些常用的**方法**：

序号	方法 & 描述
1	public override void Close() 关闭 StreamReader 对象和基础流，并释放任何与读者相关的系统资源。
2	public override int Peek() 返回下一个可用的字符，但不使用它。
3	public override int Read() 从输入流中读取下一个字符，并把字符位置往前移一个字符。

StreamWriter 类

StreamWriter 类继承自抽象类 **TextWriter**，表示编写器写入一系列字符。

下表列出了 **StreamWriter** 类中一些常用的**方法**：

序号	方法 & 描述
1	public override void Close() 关闭当前的 StreamWriter 对象和基础流。
2	public override void Flush() 清理当前编写器的所有缓冲区，使得所有缓冲数据写入基础流。
3	public virtual void Write(bool value) 把一个布尔值的文本表示形式写入到文本字符串或流。（继承自 TextWriter。）
4	public override void Write(char value) 把一个字符写入到流。
5	public virtual void Write(decimal value) 把一个十进制值的文本表示形式写入到文本字符串或流。
6	public virtual void Write(double value) 把一个 8 字节浮点值的文本表示形式写入到文本字符串或流。
7	public virtual void Write(int value) 把一个 4 字节有符号整数的文本表示形式写入到文本字符串或流。
8	public override void Write(string value) 把一个字符串写入到流。
9	public virtual void WriteLine() 把行结束符写入到文本字符串或流。

二进制文件的读写

BinaryReader 和 BinaryWriter 类用于二进制文件的读写。

BinaryReader 类

BinaryReader 类用于从文件读取二进制数据。一个 BinaryReader 对象通过向它的构造函数传递 FileStream 对象而被创建。

下表列出了 BinaryReader 类中一些常用的方法：

序号	方法 & 描述
1	public override void Close() 关闭 BinaryReader 对象和基础流。
2	public virtual int Read() 从基础流中读取字符，并把流的当前位置往前移。
3	public virtual bool ReadBoolean() 从当前流中读取一个布尔值，并把流的当前位置往前移一个字节。
4	public virtual byte ReadByte() 从当前流中读取下一个字节，并把流的当前位置往前移一个字节。
5	public virtual byte[] ReadBytes(int count) 从当前流中读取指定数目的字节到一个字节数组中，并把流的当前位置往前移指定数目的字节。
6	public virtual char ReadChar() 从当前流中读取下一个字节，并把流的当前位置按照所使用的编码和从流中读取的指定的字符往前移。
7	public virtual char[] ReadChars(int count) 从当前流中读取指定数目的字节，在一个字符数组中返回数组，并把流的当前位置按照所使用的编码和从流中读取的指定的字符往前移。
8	public virtual double ReadDouble() 从当前流中读取一个 8 字节浮点值，并把流的当前位置往前移八个字节。
9	public virtual int ReadInt32() 从当前流中读取一个 4 字节有符号整数，并把流的当前位置往前移四个字节。
10	public virtual string ReadString() 从当前流中读取一个字符串。字符串以长度作为前缀，同时编码为一个七位的整数。

BinaryWriter 类

BinaryWriter 类用于向文件写入二进制数据。一个 BinaryWriter 对象通过向它的构造函数传递 FileStream 对象而被创建。

下表列出了 BinaryWriter 类中一些常用的方法：

序号	方法 & 描述
1	public override void Close() 关闭 BinaryWriter 对象和基础流。
2	public virtual void Flush() 清理当前编写器的所有缓冲区，使得所有缓冲数据写入基础设备。
3	public virtual long Seek(int offset, SeekOrigin origin) 设置当前流内的位置。
4	public virtual void Write(bool value) 把一个单字节的布尔值写入到当前流中，0 表示 false，1 表示 true。
5	public virtual void Write(byte value) 把一个无符号字节写入到当前流中，并把流的位置往前移一个字节。
6	public virtual void Write(byte[] buffer) 把一个字节数组写入到基础流中。
7	public virtual void Write(char ch) 把一个 Unicode 字符写入到当前流中，并把流的当前位置按照所使用的编码和要写入到流中的指定的字符往前移。
8	public virtual void Write(char[] chars) 把一个字符数组写入到当前流中，并把流的当前位置按照所使用的编码和要写入到流中的指定的字符往前移。
9	public virtual void Write(double value) 把一个 8 字节浮点值写入到当前流中，并把流位置往前移八个字节。
10	public virtual void Write(int value) 把一个 4 字节有符号整数写入到当前流中，并把流位置往前移四个字节。
11	public virtual void Write(string value) 把一个以长度为前缀的字符串写入到 BinaryWriter 的当前编码的流中，并把流的当前位置按照所使用的编码和要写入到流中的指定的字符往前移。

Windows文件系统的操作

DirectoryInfo 类

DirectoryInfo 类派生自 FileSystemInfo 类。它提供了各种用于创建、移动、浏览目录和子目录的方法。该类不能被继承。

下表列出了 DirectoryInfo 类中一些常用的属性：

序号	属性 & 描述
1	Attributes 获取当前文件或目录的属性。
2	CreationTime 获取当前文件或目录的创建时间。
3	Exists 获取一个表示目录是否存在的布尔值。
4	Extension 获取表示文件存在的字符串。
5	FullName 获取目录或文件的完整路径。
6	LastAccessTime 获取当前文件或目录最后被访问的时间。
7	Name 获取该 DirectoryInfo 实例的名称。

下表列出了 DirectoryInfo 类中一些常用的方法：

序号	方法 & 描述
1	public void Create() 创建一个目录。
2	public DirectoryInfo CreateSubdirectory(string path) 在指定的路径上创建子目录。指定的路径可以是相对于 DirectoryInfo 类的实例的路径。
3	public override void Delete() 如果为空的，则删除该 DirectoryInfo。
4	public DirectoryInfo[] GetDirectories() 返回当前目录的子目录。
5	public FileInfo[] GetFiles() 从当前目录返回文件列表。

FileInfo 类

FileInfo 类派生自 **FileSystemInfo** 类。它提供了用于创建、复制、删除、移动、打开文件的属性和方法，且有助于 FileStream 对象的创建。该类不能被继承。
下表列出了 **FileInfo** 类中一些常用的**属性**：

序号	属性 & 描述
1	Attributes 获取当前文件的属性。
2	CreationTime 获取当前文件的创建时间。
3	Directory 获取文件所属目录的一个实例。
4	Exists 获取一个表示文件是否存在的布尔值。
5	Extension 获取表示文件存在的字符串。
6	FullName 获取文件的完整路径。
7	LastAccessTime 获取当前文件最后被访问的时间。
8	LastWriteTime 获取文件最后被写入的时间。
9	Length 获取当前文件的大小，以字节为单位。
10	Name 获取文件的名称。

下表列出了 **FileInfo** 类中一些常用的**方法**：

序号	方法 & 描述
1	public StreamWriter AppendText() 创建一个 StreamWriter，追加文本到由 FileInfo 的实例表示的文件中。
2	public FileStream Create() 创建一个文件。
3	public override void Delete() 永久删除一个文件。
4	public void MoveTo(string destFileName) 移动一个指定的文件到一个新的位置，提供选项来指定新的文件名。
5	public FileStream Open(FileMode mode) 以指定的模式打开一个文件。
6	public FileStream Open(FileMode mode, FileAccess access) 以指定的模式，使用 read、write 或 read/write 访问，来打开一个文件。
7	public FileStream Open(FileMode mode, FileAccess access, FileShare share) 以指定的模式，使用 read、write 或 read/write 访问，以及指定的分享选项，来打开一个文件。
8	public FileStream OpenRead() 创建一个只读的 FileStream。
9	public FileStream OpenWrite() 创建一个只写的 FileStream。

26.C#泛型（Generic）

泛型（Generic） 允许编写一个可以与任何数据类型一起工作的类或方法。
可以通过数据类型的替代参数编写类或方法的规范。当编译器遇到类的构造函数或方法的函数调用时，它会生成代码来处理指定的数据类型。

泛型（Generic）的特性

- 使用泛型是一种增强程序功能的技术，具体表现在以下几个方面：
- 它有助于最大限度地重用代码、保护类型的安全以及提高性能。
 - 可以创建泛型集合类。 .NET 框架类库在 *System.Collections.Generic* 命名空间中包含了一些新的泛型集合类。可以使用这些泛型集合类来替代 *System.Collections* 中的集合类。
 - 可以创建自己的泛型接口、泛型类、泛型方法、泛型事件和泛型委托。
 - 可以对泛型类进行约束以访问特定数据类型的方法。
 - 关于泛型数据类型中使用的类型的信息可在运行时通过使用反射获取。

泛型（Generic）方法

在上面的实例中，我们已经使用了泛型类，我们可以通过类型参数声明泛型方法

```
// 引入系统命名空间，包含基础的系统功能类
using System;

// 引入泛型集合命名空间，包含各种泛型集合类型
using System.Collections.Generic;

// 定义命名空间GenericMethodAppl，用于组织相关代码
namespace GenericMethodAppl
{
    // 定义Program类，作为程序的主类
    class Program
    {
        // 定义泛型交换方法Swap，T是类型参数
        // ref关键字表示参数按引用传递，方法内部的修改会影响外部变量
        static void Swap<T>(ref T lhs, ref T rhs)
        {
            // 声明一个T类型的临时变量temp，用于交换数据
            T temp;

            // 将lhs的值赋给temp
```

```
temp = lhs;
// 将rhs的值赋给lhs
lhs = rhs;
// 将temp（原lhs的值）赋给rhs，完成交换
rhs = temp;
}
```

```
1 // 程序入口方法Main
2 static void Main(string[] args)
3 {
4     // 声明两个int类型变量a和b
5     int a, b;
6     // 声明两个char类型变量c和d
7     char c, d;
8
9     // 给int变量a赋值10
10    a = 10;
11    // 给int变量b赋值20
12    b = 20;
13
14    // 给char变量c赋值'I'
15    c = 'I';
16    // 给char变量d赋值'V'
17    d = 'V';
18
19    // 输出交换前的int值提示信息
20    Console.WriteLine("Int values before calling swap:");
21    // 输出交换前a和b的值
22    Console.WriteLine("a = {0}, b = {1}", a, b);
23
24    // 输出交换前的char值提示信息
25    Console.WriteLine("Char values before calling swap:");
26    // 输出交换前c和d的值
27    Console.WriteLine("c = {0}, d = {1}", c, d);
28
29    // 调用泛型方法Swap，指定类型参数为int，交换a和b的值
30    Swap<int>(ref a, ref b);
31    // 调用泛型方法Swap，指定类型参数为char，交换c和d的值
32    Swap<char>(ref c, ref d);
33
34    // 输出交换后的int值提示信息
35    Console.WriteLine("Int values after calling swap:");
36    // 输出交换后a和b的值
37    Console.WriteLine("a = {0}, b = {1}", a, b);
38
39    // 输出交换后的char值提示信息
40    Console.WriteLine("Char values after calling swap:");
41    // 输出交换后c和d的值
42    Console.WriteLine("c = {0}, d = {1}", c, d);
43
44    // 等待用户按键后再关闭控制台窗口，便于查看输出结果
45    Console.ReadKey();
46 }
47 }
48 }
49
50
51 结果:
52 Int values before calling swap:
53 a = 10, b = 20
54 Char values before calling swap:
55 c = I, d = V
```

```
56 Int values after calling swap:
57 a = 20, b = 10
58 Char values after calling swap:
59 c = V, d = I
```

泛型（Generic）委托

您可以通过类型参数定义泛型委托。例如：

```
1 delegate T NumberChanger<T>(T n);
```

27.C#委托（Delegate）

在 C# 中，委托（Delegate）是一种类型安全的函数指针，它允许将方法作为参数传递给其他方法。

C# 中的委托（Delegate）类似于 C 或 C++ 中函数的指针。**委托（Delegate）** 是存有对某个方法的引用的一种引用类型变量，引用可在运行时被改变。

委托在 C# 中非常常见，用于事件处理、回调函数、LINQ 等操作。

所有的委托（Delegate）都派生自 **System.Delegate** 类。

声明委托（Delegate）

委托是一个引用类型，它定义了一个方法签名，可以用于存储指向该签名的方法。通过委托，你可以调用其他类中的方法。

委托声明决定了可由该委托引用的方法。委托可指向一个与其具有相同标签的方法。

声明委托的语法如下：

```
1 public delegate <return type> <delegate-name> <parameter list>
2 中文格式说明：
3 public delegate 返回类型 委托名(参数类型 参数名, ...);
4 例如以下代码，我们定义一个接受两个整数并返回一个整数的委托：
5 public delegate int MathOperation(int x, int y);
6 以下例子的委托可被用于引用任何一个带有一个单一的 string 参数的方法，并返回一个 int 类型变量。
7 public delegate int MyDelegate (string s);
8
```

实例化委托（Delegate）

一旦声明了委托类型，**委托对象必须使用 new 关键字来创建**，且与一个特定的方法有关。当创建委托时，传递到 **new** 语句的参数就像方法调用一样书写，但是不带有参数。例如：

```
1 public delegate void printString(string s);
2 ...
3 printString ps1 = new printString(WriteToScreen);
4 printString ps2 = new printString(WriteToFile);
```

实例：

```
1 演示了委托的声明、实例化和使用，该委托可用于引用带有一个整型参数的方法，并返回一个整型值
2 // 引入System命名空间，包含基础系统功能类
3 using System;
4
5 // 声明一个委托类型NumberChanger，它可以指向返回int且参数为int的方法
6 delegate int NumberChanger(int n);
7
8 // 定义命名空间DelegateApp1，用于组织委托相关的代码
9 namespace DelegateApp1
10 {
11     // 定义测试委托的类TestDelegate
12     class TestDelegate
```

```

13 {
14     // 声明一个静态int变量num，作为共享数据
15     static int num = 10;
16
17     // 定义静态方法AddNum，用于给num增加指定值p
18     // 方法签名与NumberChanger委托一致（返回int，参数为int）
19     public static int AddNum(int p)
20     {
21         num += p; // num = num + p
22         return num; // 返回更新后的num值
23     }
24
25     // 定义静态方法MultNum，用于给num乘以指定值q
26     // 方法签名与NumberChanger委托一致
27     public static int MultNum(int q)
28     {
29         num *= q; // num = num * q
30         return num; // 返回更新后的num值
31     }
32
33     // 定义静态方法getNum，用于获取当前num的值
34     public static int getNum()
35     {
36         return num;
37     }
38
39     // 程序入口方法Main
40     static void Main(string[] args)
41     {
42         // 创建委托实例nc1，关联到AddNum方法
43         NumberChanger nc1 = new NumberChanger(AddNum);
44         // 创建委托实例nc2，关联到MultNum方法
45         NumberChanger nc2 = new NumberChanger(MultNum);
46
47         // 通过委托nc1调用AddNum方法，传入参数25
48         // 此时会执行num += 25（num初始为10，结果为35）
49         nc1(25);
50         // 输出当前num的值（35）
51         Console.WriteLine("Value of Num: {0}", getNum());
52
53         // 通过委托nc2调用MultNum方法，传入参数5
54         // 此时会执行num *= 5（35 * 5 = 175）
55         nc2(5);
56         // 输出当前num的值（175）
57         Console.WriteLine("Value of Num: {0}", getNum());
58
59         // 等待用户按键后关闭控制台窗口，方便查看结果
60         Console.ReadKey();
61     }
62 }
63 }
64
65 当上面的代码被编译和执行时，它会产生下列结果：
66 Value of Num: 35
67 Value of Num: 175
68
69

```

委托的多播（Multicasting of a Delegate）

委托对象可使用 `+` 运算符进行合并。

一个合并委托调用它所合并的两个委托，只有相同类型的委托可被合并。

■ 运算符可用于从合并的委托中移除组件委托。

使用委托的这个有用的特点，可以创建一个委托被调用时要调用的方法的调用列表，这被称为委托的 **多播（multicasting）**，也叫组播。

委托（Delegate）的用途

委托 *printString* 可用于引用带有一个字符串作为输入的方法，并不返回任何东西。

我们使用这个委托来调用两个方法，第一个把字符串打印到控制台，第二个把字符串打印到文件：

```
1 // 引入System命名空间，提供基础系统功能
2 using System;
3 // 引入System.IO命名空间，提供文件输入输出功能
4 using System.IO;
5
6 // 定义命名空间DelegateApp1，用于组织相关代码
7 namespace DelegateApp1
8 {
9     // 定义PrintString类，包含委托及相关方法
10    class PrintString
11    {
12        // 声明静态FileStream对象fs，用于文件操作
13        static FileStream fs;
14        // 声明静态StreamWriter对象sw，用于写入文件
15        static StreamWriter sw;
16
17        // 声明委托类型printString，它可以指向参数为string且无返回值的方法
18        public delegate void printString(string s);
19
20        // 定义静态方法WriteToScreen，用于将字符串打印到控制台
21        // 方法签名与printString委托一致（参数为string，无返回值）
22        public static void WriteToScreen(string str)
23        {
24            // 在控制台输出格式化字符串
25            Console.WriteLine("The String is: {0}", str);
26        }
27
28        // 定义静态方法WriteToFile，用于将字符串写入文件
29        // 方法签名与printString委托一致
30        public static void WriteToFile(string s)
31        {
32            // 创建或打开文件c:\message.txt，以追加模式写入
33            fs = new FileStream("c:\\message.txt", FileMode.Append, FileAccess.Write);
34            // 创建StreamWriter对象，用于向文件写入内容
35            sw = new StreamWriter(fs);
36            // 将字符串s写入文件
37            sw.WriteLine(s);
38            // 清空缓冲区，确保数据写入文件
39            sw.Flush();
40            // 关闭StreamWriter
41            sw.Close();
42            // 关闭FileStream
43            fs.Close();
44        }
45
46        // 定义静态方法sendString，接收printString类型的委托作为参数
47        public static void sendString(printString ps)
48        {
49            // 通过委托参数调用关联的方法，传入"Hello World"作为参数
50            ps("Hello World");
51        }
52
53        // 程序入口方法Main
54        static void Main(string[] args)
55        {
56            // 创建委托实例ps1，关联到WriteToScreen方法
```

```

57     printString ps1 = new printString(WriteToScreen);
58     // 创建委托实例ps2, 关联到WriteToFile方法
59     printString ps2 = new printString(WriteToFile);
60
61     // 调用sendString方法, 传入ps1委托, 会执行WriteToScreen("Hello World")
62     sendString(ps1);
63     // 调用sendString方法, 传入ps2委托, 会执行WriteToFile("Hello World")
64     sendString(ps2);
65
66     // 等待用户按键后关闭控制台窗口, 方便查看结果
67     Console.ReadKey();
68 }
69 }
70 }
71
72
73 结果:
74 The String is: Hello World

```

移除委托

如果你不再需要某个方法, 可以通过 `-=` 运算符将该方法从委托链中移除。

委托和事件

委托常常与事件 (Event) 一起使用, 事件是一种特殊类型的委托, 用于发布和订阅机制。

在 C# 中, 事件本质上就是一个封装了委托的类型, 它用于响应程序中的某些操作。

C#中常见的委托类型:

1、Action

Action: 代表不返回值的方法。可以接受最多 16 个参数。

```

1 Action<string> printMessage = Console.WriteLine;
2 printMessage("Hello");

```

2、Func

Func: 代表有返回值的方法。最多接受 16 个参数, 第一个参数是输入参数, 最后一个参数是返回值类型。

```

1 Func<int, int, int> add = (x, y) => x + y;
2 Console.WriteLine(add(3, 4)); // 输出 7

```

3、Predicate

Predicate: 代表返回 bool 值的方法, 通常用于条件判断。

```

1 Predicate<int> isEven = x => x % 2 == 0;
2 Console.WriteLine(isEven(4)); // 输出 True

```

类型安全: 委托是类型安全的, 这意味着只有签名匹配的方法才能赋值给委托。

匿名方法和 lambda 表达式: 你可以使用匿名方法或 lambda 表达式来创建委托实例, 简化代码。

```

1 Func<int, int, int> add = (x, y) => x + y;
2 Console.WriteLine(add(5, 3)); // 输出 8

```

异步调用: 可以将委托与 BeginInvoke 和 EndInvoke 方法一起使用, 进行异步调用。

28.C#事件（Event）

C# 事件（Event）是一种成员，用于将特定的事件通知发送给订阅者。

事件通常用于实现观察者模式，它允许一个对象将状态的变化通知其他对象，而不需要知道这些对象的细节。

事件（Event）基本上说是一个**用户操作**，如**按键**、**点击**、**鼠标移动**等等，或者是一些提示信息，如系统生成的通知。**应用程序需要在事件发生时响应事件**。例如，中断。

关键点：

- **声明委托**：定义事件将使用的委托类型。委托是一个函数签名。
- **声明事件**：使用 `event` 关键字声明一个事件。
- **触发事件**：在适当的时候调用事件，通知所有订阅者。
- **订阅和取消订阅事件**：其他类可以通过 `+=` 和 `-=` 运算符订阅和取消订阅事件。

通过事件使用委托

事件在类中声明且生成，且通过使用同一个类或其他类中的委托与事件处理程序关联。包含事件的类用于发布事件。这被称为 **发布者（publisher）** 类。其他接受该事件的类被称为 **订阅器（subscriber）** 类。事件使用 **发布-订阅（publisher-subscriber）** 模型。

发布者（publisher） 是一个包含事件和委托定义的对象。事件和委托之间的联系也定义在这个对象中。发布者（publisher）类的对象调用这个事件，并通知其他的对象。

订阅器（subscriber） 是一个接受事件并提供事件处理程序的对象。在发布者（publisher）类中的委托调用订阅器（subscriber）类中的方法（事件处理程序）。

声明事件（Event）

在类的内部声明事件，首先必须声明该事件的委托类型。例如：

```
1 public delegate void BoilerLogHandler(string status);
```

然后，声明事件本身，使用 **event** 关键字：

```
1 // 基于上面的委托定义事件
2 public event BoilerLogHandler BoilerEventLog;
```

上面的代码定义了一个名为 *BoilerLogHandler* 的委托和一个名为 *BoilerEventLog* 的事件，该事件在生成的时候会调用委托。

例子：

// 引入System命名空间，提供基础系统功能

using System;

// 定义命名空间EventDemo，用于组织事件相关代码

namespace EventDemo

{

// 定义委托类型NotifyEventHandler，作为事件处理程序的类型

// 遵循.NET事件委托标准：参数为object sender（事件发送者）和EventArgs e（事件数据）

public delegate void NotifyEventHandler(object sender, EventArgs e);

```
1 // 引入System命名空间，提供基础系统功能
2 using System;
3
4 // 定义命名空间EventDemo，用于组织事件相关代码
5 namespace EventDemo
6 {
7     // 定义委托类型NotifyEventHandler，作为事件处理程序的类型
8     // 遵循.NET事件委托标准：参数为object sender（事件发送者）和EventArgs e（事件数据）
9     public delegate void NotifyEventHandler(object sender, EventArgs e);
10
11 // 发布者类（事件的发送方），包含业务逻辑并触发事件
12 public class ProcessBusinessLogic
13 {
14     // 声明事件ProcessCompleted，类型为NotifyEventHandler
15     // 事件本质是特殊的委托，只能在类内部触发，外部只能订阅/取消订阅
16     public event NotifyEventHandler ProcessCompleted;
17
18     // 受保护的虚拟方法，用于触发事件（遵循.NET事件命名规范：On+事件名）
19     // 虚拟方法允许子类重写事件触发逻辑
20     protected virtual void OnProcessCompleted(EventArgs e)
21     {
```

```

22     // 检查是否有订阅者，若有则触发事件
23     // ?. 是null条件运算符，避免ProcessCompleted为null时调用Invoke报错
24     // 传递当前实例(this)作为发送者，空事件数据(EventArgs.Empty)
25     ProcessCompleted?.Invoke(this, e);
26 }
27
28 // 公开方法，模拟业务逻辑处理并在完成后触发事件
29 public void StartProcess()
30 {
31     Console.WriteLine("Process Started!"); // 输出提示：流程已开始
32
33     // 这里可以加入实际的业务逻辑代码（如数据处理、文件操作等）
34
35     // 业务逻辑完成后，调用OnProcessCompleted方法触发事件
36     OnProcessCompleted(EventArgs.Empty);
37 }
38 }
39
40 // 订阅者类（事件的接收方），响应事件并执行相应操作
41 public class EventSubscriber
42 {
43     // 订阅事件的方法：将事件处理程序绑定到发布者的事件
44     public void Subscribe(ProcessBusinessLogic process)
45     {
46         // 为ProcessCompleted事件添加处理方法Process_ProcessCompleted
47         process.ProcessCompleted += Process_ProcessCompleted;
48     }
49
50     // 事件处理程序：当事件触发时执行的逻辑
51     // 方法签名必须与NotifyEventHandler委托一致
52     private void Process_ProcessCompleted(object sender, EventArgs e)
53     {
54         Console.WriteLine("Process Completed!"); // 输出提示：流程已完成
55     }
56 }
57
58 // 程序主类
59 class Program
60 {
61     // 程序入口方法
62     static void Main(string[] args)
63     {
64         // 创建发布者实例（业务逻辑处理器）
65         ProcessBusinessLogic process = new ProcessBusinessLogic();
66         // 创建订阅者实例
67         EventSubscriber subscriber = new EventSubscriber();
68
69         // 订阅事件：让订阅者关注发布者的ProcessCompleted事件
70         subscriber.Subscribe(process);
71
72         // 启动业务流程，执行完成后会自动触发事件
73         process.StartProcess();
74
75         // 等待用户输入，防止控制台窗口立即关闭
76         Console.ReadLine();
77     }
78 }
79 }

```

29.C#集合 (Collection)

集合 (Collection) 类是专门用于数据存储和检索的类。这些类提供了对栈 (stack)、队列 (queue)、列表 (list) 和哈希表 (hash table) 的支持。大多数集合类实现了相同的接口。

集合（Collection）类服务于不同的目的，如为元素动态分配内存，基于索引访问列表项等等。这些类创建 `Object` 类的对象的集合。在 C# 中，`Object` 类是所有数据类型的基类。接口。

集合类：

类	描述和用法
动态数组（ArrayList）	它代表了可被单独索引的对象的有序集合。它基本上可以替代一个数组。但是，与数组不同的是，您可以使用索引在位置添加和移除项目，动态数组会自动重新调整它的大小。它也允许在列行动态内存分配、增加、搜索、排序各项。
哈希表（Hashtable）	它使用键来访问集合中的元素。当您使用键访问元素时，则使用哈希表，而且您可以识别一个有用的键值表中的每一项都有一个键/值对。键用于访问集合中的项目。
排序列表（SortedList）	它可以使用键和索引来访问列表中的项。排序列表是数组和哈希表的组合。它包含一个可使用键或索引访问各项的如果您使用索引访问各项，则它是一个动态数组（ArrayList），如果您使用键访问各项，则它是一个哈希表（Hashtable）。集合中的各项总是按键值排序。
堆栈（Stack）	它代表了一个后进先出的对象集合。当您需要对各项进行后进先出的访问时，则使用堆栈。当您在列表中添加称为推入元素，当您从列表中移除一项时，称为弹出元素。
队列（Queue）	它代表了一个先进先出的对象集合。当您需要对各项进行先进先出的访问时，则使用队列。当您在列表中添加称为入队，当您从列表中移除一项时，称为出队。
点阵列（BitArray）	它代表了一个使用值 1 和 0 来表示的二进制数组。当您需要存储位，但是事先不知道位数时，则使用点阵列。您可以使用整从点阵列集合中访问各项，索引从零开始。

C#动态数组（ArrayList）

ArrayList 是C#中提供的一种动态数组类，位于命名空间System.Collection中。

动态数组与普通数组不同，它的大小可以动态调整，无需预先定义固定长度。

动态数组代表了可以被单独索引的对象的有序集合，它允许在列表中进行动态内存分配，增加，搜索，排序各项。

动态数组 `ArrayList` 类适用于需要频繁调整大小的数组操作，但由于其非泛型特性，在现代 C# 开发中逐渐被 `List<T>` 替代。

基本结构

```
1 public class ArrayList : ICollection, IEnumerable, IList, ICloneable
2
3 实现接口：
4 IList：支持按索引访问元素。
5 ICollection：支持集合基本操作，如添加、删除和计数。
6 IEnumerable：支持通过枚举器遍历元素。
7 ICloneable：支持克隆 ArrayList。
```

特点

动态扩展：

- `ArrayList` 的容量可以根据需要自动调整，不需要指定固定大小。
- 当添加的元素超过当前容量时，`ArrayList` 会自动增加容量（通常是原容量的两倍）。

非泛型集合：

- `ArrayList` 是非泛型集合，所有元素被存储为 `object` 类型。这意味着它可以存储任意类型的对象，但需要注意装箱（boxing）和拆箱（unboxing）的性能影响。

无序操作：

- 虽然元素存储顺序与添加顺序一致，但它并不提供内置排序功能。

线程安全：

- 默认不是线程安全的。如果需要线程安全的 `ArrayList`，可以使用 `ArrayList.Synchronized` 方法生成一个线程安全的版本。

ArrayList 类的方法和属性

链接：[C# 动态数组（ArrayList）](#) | [菜鸟教程](#)

ArrayList与泛型集合 List<T> 的对比

特性	ArrayList	List<T>
类型安全	否（存储 <code>object</code> 类型）	是（泛型，强类型约束）
装箱与拆箱	是（值类型需要）	否
性能	较低	较高
泛型支持	无	有
使用场景	遗留代码或简单场景	现代开发中优先选择

注意事项

避免频繁装箱和拆箱：

- 如果数据主要是值类型，建议使用 `List<T>` 来避免性能开销。

慎用混合类型数据：

- 尽量不要混合存储不同类型的对象，以降低运行时错误的风险。

替代方案：

- 在现代 C# 开发中，推荐使用泛型集合（如 `List<T>`、`Dictionary<TKey, TValue>`）以提高代码的安全性和效率。

C#哈希表（Hashtable）

哈希表（Hashtable）是一种以键值对（`key=>value`）形式存储数据的集合，键和值可以是任意对象。

哈希表中的每一项都有一个`key=>value`对，key 用于访问集合中的项目。

哈希表基于哈希算法组织和访问数据，提供高效的查找、插入和删除操作。

`Hashtable` 是非泛型集合，位于命名空间 `System.Collections` 中。如果需要泛型版本，可以使用 `Dictionary<TKey, TValue>`。

基本结构：

- 键**（`Key`）：用于标识每个数据项，必须是唯一的，不能为 `null`。
- 值**（`Value`）：键所对应的数据，可以为 `null`。

特点：

- 键值对存储**：每个键都是唯一的，与一个值相关联。
- 哈希算法**：通过键的哈希码定位数据存储位置，查找和插入操作的平均时间复杂度为 O(1)。
- 动态调整容量**：当哈希表中元素数目超过容量时，表会动态扩展并重新分配存储。
- 无序存储**：键值对的存储顺序与插入顺序无关，由哈希码决定。

Hashtable 类的方法和属性

链接：[C# 哈希表（Hashtable） | 菜鸟教程](#)

与其他集合的对比

特性	Hashtable	Dictionary<TKey, TValue>
类型安全	否（非泛型，存储 <code>object</code> 类型）	是（泛型，强类型约束）
性能	较慢（需进行装箱和拆箱操作）	更快（无装箱操作，泛型直接支持类型）
键的唯一性	是	是
值是否可为 <code>null</code>	是	是
排序	无	无
线程安全	默认非线程安全	默认非线程安全

- `Hashtable` 是一种高效的键值对集合，适用于需要快速存储和检索键值对的场景。
- 非泛型的设计使其在类型安全和性能方面逊色于泛型版本的集合（如 `Dictionary<TKey, TValue>`）。
- 在现代 C# 开发中，`Dictionary<TKey, TValue>` 更常被使用，但 `Hashtable` 仍然适合简单应用或与遗留代码兼容的场景。

C#排序列表（SortedList）

排序列表（SortedList）是一种按键自动排序的集合，它通过键值对存储数据，并在添加或更新时根据键的顺序（升序）自动排序。

SortedList 类代表了一系列按照键来排序的 `key => value` 对，这些键值对可以通过键和索引来访问。

排序列表是数组和哈希表的组合。它包含一个可使用键或索引访问各项的列表。如果您使用索引访问各项，则它是一个动态数组（ArrayList），如果您使用键访问各项，则它是一个哈希表（Hashtable）。集合中的各项总是按键值排序。

SortedList 有两种版本：

- 1. 非泛型 `SortedList` (`System.Collections.SortedList`)
- 2. 泛型 `SortedList<TKey, TValue>` (`System.Collections.Generic.SortedList<TKey, TValue>`)

特点:

- **自动排序**：所有键都按照升序排序，无需手动调用排序方法。
- **键值对存储**：每个键都唯一，与其关联的值可以重复。
- **动态调整容量**：根据需要自动调整内部存储的大小。
- **索引访问**：通过键或索引访问数据。
- **高效操作**：检索、插入、删除的平均时间复杂度为 $O(\log n)$ 。

SortedList 类的方法和属性

链接：[C# 排序列表 \(SortedList\)](#) | [菜鸟教程](#)

30.C#多线程

线程 被定义为程序的执行路径。每个线程都定义了一个独特的控制流。如果您的应用程序涉及到复杂的和耗时的操作，那么设置不同的线程执行路径往往是有益的，每个线程执行特定的工作。

线程是**轻量级进程**。一个使用线程的常见实例是现代操作系统中并行编程的实现。使用线程节省了 CPU 周期的浪费，同时提高了应用程序的效率。

到目前为止我们编写的程序是一个单线程作为应用程序的运行实例的单一的过程运行的。但是，这样子应用程序同时只能执行一个任务。为了同时执行多个任务，它可以被划分为更小的线程。

线程生命周期

线程生命周期开始于 System.Threading.Thread 类的对象被创建时，结束于线程被终止或完成执行时。

下面列出了线程生命周期中的各种状态：

- **未启动状态**：当线程实例被创建但 Start 方法未被调用时的状况。
- **就绪状态**：当线程准备好运行并等待 CPU 周期时的状况。
- **不可运行状态**：下面的几种情况下线程是不可运行的：
 - 已经调用 Sleep 方法
 - 已经调用 Wait 方法
 - 通过 I/O 操作阻塞
- **死亡状态**：当线程已完成执行或已中止时的状况。

主程序

C#中，**System.Treading.Thread**类用于线程的工作。它允许创建并访问多线程应用程序中的单个线程。进程中第一个被执行的线程被称为**主线程**。

C#程序开始执行时，主线程自动创建，使用**Thread**类创建的线程被主线程的子线程调用，可以使用Thread类的**CurrentTread**属性访问线程。

Thread类的常用方法：

链接：[C# 多线程](#) | [菜鸟教程](#)

创建线程

线程是通过扩展 Thread 类创建的。扩展的 Thread 类调用 **Start()** 方法来开始子线程的执行。

管理线程

Thread 类提供了各种管理线程的方法。

下面的实例演示了 **sleep()** 方法的使用，用于在一个特定的时间暂停线程。

```
1 using System;
2 using System.Threading;
3 namespace MultithreadingApplication
4 {
5     class ThreadCreationProgram
6     {
7         public static void CallToChildThread()
8         {
9             Console.WriteLine("Child thread starts");
10            // 线程暂停 5000 毫秒
```

```

11         int sleepfor = 5000;
12         Console.WriteLine("Child Thread Paused for {0} seconds",
13                             sleepfor / 1000);
14         Thread.Sleep(sleepfor);
15         Console.WriteLine("Child thread resumes");
16     }
17     static void Main(string[] args)
18     {
19         ThreadStart childref = new ThreadStart(CallToChildThread);
20         Console.WriteLine("In Main: Creating the Child thread");
21         Thread childThread = new Thread(childref);
22         childThread.Start();
23         Console.ReadKey();
24     }
25 }
26 }

```

销毁线程

Abort() 方法用于销毁线程。

通过抛出 **threadabortexception** 在运行时中止线程。这个异常不能被捕获，如果有 *finally* 块，控制会被送至 *finally* 块。

```

1  using System;
2  using System.Diagnostics.Metrics;
3  using System.Threading;
4
5  namespace MultithreadingApplication
6  {
7      class ThreadCreationProgram
8      {
9          public static void CallToChildThread()
10         {
11             try
12             {
13                 Console.WriteLine("Child thread starts");
14                 //计数到10
15                 for(int counter = 0; counter <= 10; counter++)
16                 {
17                     Thread.Sleep(500);
18                     Console.WriteLine(counter);
19
20                 }
21                 Console.WriteLine("Child Thread Completed");
22
23             }
24             catch (ThreadAbortException e)
25             {
26                 Console.WriteLine("Thread Abort Exception");
27             }
28             finally
29             {
30                 Console.WriteLine("Couldn't catch the Thread exception");
31             }
32         }
33
34         static void Main(string[] args)
35         {
36             ThreadStart childref = new ThreadStart(CallToChildThread);
37             Console.WriteLine("In Main:Creating the Child thread");
38             Thread childThread = new Thread(childref);
39             childThread.Start();
40             //停止主线程一段时间

```



```
41     Thread.Sleep(2000);
42     //中止子线程
43     Console.WriteLine("In Main:Aborting the Child thread");
44     childThread.Abort();
45     Console.ReadKey();
46
47 }
48
49 }
```

C#高级